

Chapter 1

INTRODUCTION

1.1 Background and Basics

The administration and management of large educational institutions today necessitate a digital transformation away from fragmented, paper-based, and manual processes. Traditional systems struggle to keep pace with the complex, multi-tiered hierarchy of modern colleges, which involves administrators, principals, department heads, faculty, and a large student body. This inefficiency often leads to delays in critical operations, such as event and club approvals, and results in poor transparency regarding student progress in mandatory academic sections like projects, seminars, and internships. Moreover, communication, which is the backbone of a vibrant campus, often lacks immediacy, with reliance on asynchronous methods like email or physical notice boards. To address these gaps, a comprehensive Campus Management System is required—a singular, centralized platform designed to manage institutional data, academic workflows, and campus communication seamlessly. This proposed system aims to integrate all user functionalities, from the Campus Admin's ability to create colleges and assign principals, to the Student's need to view timetables and submit coursework. This proposed system aims to integrate all user functionalities, from the Campus Admin's ability to create colleges and assign principals, to the Student's need to view timetables and submit coursework. The platform incorporates real-time push notifications via Firebase Cloud Messaging (FCM) and asynchronous email alerts via Spring Mail, enabling immediate acknowledgement of critical operations such as ticket confirmation, club approvals, and broadcast delivery — fundamentally improving the speed and effectiveness of day-to-day campus operations.

1.2 Literature Survey

The design of the Campus Management System (CMS) aims to solve the problem of fragmented and inefficient academic and administrative processes within higher education institutions. This project is characterized by three critical domains: Multi-Tiered Access Control and Hierarchy Management, Digital Workflow Automation, and Real-Time Data Synchronization for Academic Monitoring. A review of existing literature reveals established technical approaches used to tackle these challenges.

1.2.1 Hierarchical Access Control and Role-Based Management

Effective campus management requires a system that accurately reflects the administrative and academic hierarchy—from the Campus Admin down to the Student. Existing information systems in large organizations typically utilize layered security models to manage permissions.

- **Role-Based Access Control (RBAC):** RBAC is the foundational approach for managing permissions in complex organizational structures. In RBAC models, access is granted or

denied based on the user's role (e.g., Principal, HOD, Professor) rather than the individual user's identity. This approach simplifies administrative tasks by allowing the Campus Admin to assign the Principal role, and the Principal to assign the HOD role, without manually defining every user's individual permissions. Literature emphasizes that RBAC reduces administrative overhead, improves policy enforcement, and minimizes security risks by limiting users to the exact permissions necessary for their tasks (e.g., a Professor can access student progress records but cannot modify the department structure).

- **Multi-Tiered Architecture:** Existing large-scale enterprise solutions often rely on a threetier or n-tier architecture (Presentation, Business Logic, and Data layers). This segregation ensures that the complex hierarchical logic—such as restricting a Professor to managing only their assigned batches, or limiting a Principal's authority to their college's departments—resides securely in the Business Logic Layer. This approach ensures scalability and maintainability, allowing the system to handle the increasing volume of users and features required for a full campus deployment.
- **Delegated Administration:** To manage the workload, systems employ delegated administration, where high-level users grant specific management privileges to subordinates. In this project, the Campus Admin delegates authority to Principals, who, in turn, delegate authority to HODs (e.g., the authority to approve club requests or manage private seminar halls within their domain). This model prevents administrative bottlenecks, allowing management tasks to be handled efficiently at the most relevant organizational level.

1.2.2 Workflow Automation and Event Management

Many core features of the CMS, such as club formation, event approval, and student project tracking, involve a sequence of steps requiring different actors to perform specific actions. This challenge is typically solved through digital workflow management systems.

- **Sequential Workflow Models:** The established method for managing processes like the Student Club Creation involves a sequential workflow. The flow is predefined (Student submission $\rightarrow$ Professor/Guide endorsement $\rightarrow$ HOD approval $\rightarrow$ Principal final approval). Workflow engines in existing solutions manage state transitions, ensuring that a request cannot skip steps or be approved out of order. This approach enforces compliance and transparency, providing all participants with real-time status updates.
- **Event Lifecycle Management:** Existing systems use event-driven architectures to manage public and private campus events. The process includes creation (by Campus Admin for main events, or Teacher/Club for sub-events), approval (Principal/HOD), broadcast, registration, and payment. Literature highlights that integrating a Payment Gateway and an automated Ticket Generation service into the event creation workflow dramatically reduces administrative effort, turning a complex manual process into a single, seamless digital transaction.
- **Digital Form and Document Management:** Academic workflows require the handling of structured data, such as student submissions for Project, Seminar, and Internship sections. Existing approaches replace paper-based forms with digital submission portals. This allows Professors to access, review, and give remarks directly through the interface,

ensuring that all academic records are instantly logged in a centralized database, facilitating the student's ability to view their progress dashboard in real time.

1.2.3 Real-Time Data Synchronization and Academic Monitoring

The utility of the CMS hinges on its ability to provide immediate feedback and up-to-date information, particularly for academic records and broad communication.

- **Push-based Notification Systems:** For high-priority communication like campus-level broadcast messages or emergency alerts, systems rely on push-based notification services. Unlike traditional polling mechanisms, push systems deliver data from the server to the client immediately upon update. This ensures that users, whether they are monitoring data as a Campus Admin or simply viewing a newly uploaded note as a Student, receive time-sensitive information without delay. This approach supports the "always-on" expectation for critical campus communication.
- **Centralized Data Repository for Academic Tracking:** Solving the problem of fragmented student academic progress monitoring requires a centralized, nonredundant database architecture. Existing educational data systems employ data modeling techniques to link a student's record to their enrollment, course, and specific Project, Seminar, and Internship status records. This model ensures that when a Professor submits a remark for a team's project, the update instantly propagates and is available across the student's profile and the HOD's monitoring view.
- **User-Centric Dashboard Design:** Literature on educational technology emphasizes that the value of real-time data is maximized through user-centric dashboards. The system requires specialized interface designs for each role: the Campus Admin's dashboard focuses on monitoring data and high-level approvals; the Professor's dashboard prioritizes checking status of submissions and marking availability; and the Student's dashboard focuses on viewing timetable, progress, and joining events. This design philosophy ensures that each user's primary task is immediately accessible and actionable within the system.

1.3 Project Undertaken

The aim of the project is to build a unified, web-based Campus Management System (CMS) with advanced, role-specific features that address critical pain points in campus administration, academic tracking, and communication. The CMS's features solve issues identified in traditional manual or fragmented digital processes, thereby making the operation and monitoring of educational institutions more efficient and transparent for all users (Campus Admin, Principal, HOD, Professor, and Student).

1.3.1 Problem Definition

To build a **Campus Management System** that centralizes institutional control, streamlines multi-level administrative workflows, and provides real-time academic progress tracking, thereby overcoming the challenges of:

- **Decentralized Hierarchy:** Lack of a structured digital mechanism for the Campus Admin to create colleges with auto-generated unique codes, assign Principals, and for Principals to assign HODs to specific departments.
- **Inefficient Approvals:** Cumbersome, sequential paperwork required for processes like Club Creation (PENDING_HOD → PENDING_PRINCIPAL → APPROVED/REJECTED) and Event Approval, leading to delays and lack of transparency.
- **Fragmented Academic Oversight:** Difficulty for Professors and Students to track mandatory academic sections (Project, Seminar, Internship), centralize file submissions, and record feedback (remarks) with status updates (SUBMITTED → UNDER_REVIEW → APPROVED/REJECTED).
- **Manual Revenue Collection:** Labor-intensive processes for managing event registration, ticket generation, and payment — replaced by Stripe PaymentIntent integration with webhook-based ticket confirmation.

1.3.2 Scope Statement

The scope of the product is to build a CMS platform with fundamental features (user authentication, profile management, broad messaging) along with **five critical, role-specific features**. These advanced features will empower the platform and make it more efficient across administrative, academic, and event management functionalities.

The critical features are as follows:

- **Multi-Level Role Assignment and Hierarchy Management:** The Campus Admin creates colleges with auto-generated unique codes and assigns the Principal role. The Principal creates departments and assigns HODs. Spring Security 7 enforces role-based URL-pattern restrictions so each of the five roles — CAMPUS_ADMIN, PRINCIPAL, HOD, PROFESSOR, STUDENT — can only access their designated endpoints.

- **Workflow-Enabled Club and Event Approval:** Club formation follows a two-stage finite-state machine: the Student submits a request (PENDING_HOD), the HOD approves or rejects it, and if approved the status advances to PENDING_PRINCIPAL for final sanction. Invalid state transitions are rejected server-side with HTTP 400. Events are approved by HOD or Principal depending on their level (DEPARTMENT or COLLEGE).
- **Three-Section Academic Progress Tracker:** Professors manage student submissions across three BatchSection types — PROJECT, SEMINAR, and INTERNSHIP. Each section has a deadline. Professors review submissions and record remarks with a SubmissionStatus (UNDER_REVIEW, APPROVED, REJECTED). Students view their progress in real time via GET /api/student/progress.
- **Event Ticket Generation and Payment Management:** Paid events trigger a Stripe PaymentIntent on the backend; the client secret is returned to the frontend for Stripe.js payment. A webhook at /api/webhook/stripe listens for payment_intent.succeeded and confirms the ticket status from PENDING to CONFIRMED, followed by an FCM push notification to the student's device.
- **Time-Based Teacher Availability Marking:** Professors mark availability slots with date, from/to time, and a status of AVAILABLE, BUSY, or ON_LEAVE with an optional note. Students view all teachers' availability via GET /api/student/teacher-availability to plan appointments and project discussions.

1.4 Organization of Project Report

This report is organized as follows:

- Chapter 1: INTRODUCTION gives the Introduction to the Background and Basics of Campus Management, the Literature Survey conducted on multi-role systems and realtime architecture, and details about the Project to be undertaken, including the Problem Definition and Scope Statement of the CMS.
- Chapter 2: PROJECT PLANNING AND MANAGEMENT gives an overview of Project Planning and Management. It states the detailed System Requirement Specification (SRS), covering Functional Requirements (role-specific features), NonFunctional Requirements (security, performance), Deployment Requirements, External Interface Requirements, and Other Requirements. This chapter also specifies the Project Process Model used, Cost and Effort estimates, and Project Scheduling.
- Chapter 3: ANALYSIS AND DESIGN gives the Analysis Study and Designing of the Project. It consists of the IDEA Matrix, Mathematical Model, Feasibility Analysis, Architecture Diagram, and all UML diagrams (Use-Case, Activity, Class, Sequence/DFD, Component/Interface, and State Machine diagrams) specifying how to build the required multi-role system.
- Chapter 4: IMPLEMENTATION AND CODING describes the development approach, technologies employed, and key coding structures. It covers the backend built with

Spring Boot 4.0.3 (Java 17, PostgreSQL), the React 18 + Vite frontend, the JWT-based security layer, all major classes and their responsibilities across 88 REST endpoints, the Stripe payment and Firebase integrations, and the code listing with screenshots.

- Chapter 5: TESTING gives the test cases identified to perform at the final stage. This includes detailed test cases for Unit Testing, test cases for Integration Testing (intermodule communication), and test cases for Acceptance Testing (functional and nonfunctional validation).
- Chapter 6: CONCLUSION gives the final Conclusion of the project work, summarizing the successful implementation of the multi-role CMS.

Chapter 2

PROJECT PLANNING AND MANAGEMENT

2.1 Introduction

This chapter covers the project planning and management details for the Campus Management System (CMS). It outlines the foundational framework used to guide the development process, including the selection of the appropriate methodology and the estimation of necessary resources. Crucially, this section also details the System Requirement Specification (SRS), which formally documents all functional and non-functional requirements. The SRS is considered as the base for the effort estimations and project scheduling, ensuring that the project timeline and resource allocation are accurately aligned with the complexity and scope of the desired multi-role system.

2.2 System Requirement Specification (SRS)

This section gives the detailed description of all types of requirements to be satisfied by the System to be developed, the Campus Management System (CMS). The SRS establishes the foundation for the complex, multi-tiered hierarchy and workflows specific to campus operations.

2.2.1 System Overview

The system to be developed is a Unified Web-Based Campus Management Platform with advanced, role-specific features. The system manages five primary user roles: CAMPUS_ADMIN, PRINCIPAL, HOD, PROFESSOR, and STUDENT.

The backend is built with Spring Boot 4.0.3 and Java 17, following a strict layered architecture: Controller → Service Interface → Service Implementation → Repository → PostgreSQL. Authentication is handled through stateless JWT tokens (jjwt 0.12.6) validated by Spring Security 7. On every request, JwtAuthFilter extracts the Bearer token, validates its signature and expiry, and checks the InvalidatedToken table for blacklisted JTIs. Large files (professor notes, student submissions, club logos, event posters) are stored in Firebase Storage and the returned public URL is persisted in the database. Push notifications are delivered via Firebase Cloud Messaging (FCM) and emails via Spring Mail with @Async. Paid event transactions are processed through the Stripe Java SDK v31 using the StripeClient API. The frontend is built with React 18 and Vite, with Zustand for client state, React Query for server state and caching, Axios for HTTP with interceptors, and React Hook Form + Zod for form validation.

2.2.2 Functional Requirements

The features described below are organized based on their context of functionality and align with the core roles. All the features have equal priority.

2.2.2.1 Multi-Level Role Assignment and Hierarchy Management

Description and Priority: Traditional management lacks a defined digital hierarchy, making role assignment and permission management cumbersome. This feature provides the Campus Admin with ultimate control to digitally structure the institution, delegating management authority down the hierarchy (Principal -> HOD -> Professor).

Stimulus/Response Sequence: When the Campus Admin registers a new college, they assign a Professor the Principal role. Subsequently, the Principal can promote a Professor to HOD for a specific department. These actions instantly update the user's access permissions and dashboard views.

Functional Requirements:

- The Campus Admin must be able to create a college record with an auto-generated unique college code and assign a Principal from registered users via email lookup.
- The Campus Admin must be able to toggle college status (PENDING → ACTIVE → INACTIVE) through dedicated endpoints.
- The Principal must be able to create departments under their college (collegeId is auto-injected from their profile — not passed by the frontend) and assign the HOD role using a searchable user dropdown.
- User permissions must be enforced at the URL-pattern level by Spring Security 7, with role-specific base paths: /api/admin, /api/principal, /api/hod, /api/professor, /api/student.

Main Flow:

- Campus Admin authenticates and calls POST /api/admin/colleges with college name, address, city, state, and logo URL. CollegeServiceImpl auto-generates a unique college code.
- Campus Admin calls PUT /api/admin/colleges/{id}/assign-principal with the principal's email. The service validates the user has the PRINCIPAL role and associates them with the college.
- Principal logs in and calls POST /api/principal/departments. The controller auto-injects the Principal's collegeId from their authenticated profile.
- Principal calls PUT /api/principal/departments/{id}/assign-hod with a userId. The service validates the HOD role and updates the department record.
- Spring Security immediately enforces the updated role on all subsequent requests via JwtAuthFilter and the SecurityContext.

Exceptional Flow:

- If the Campus Admin attempts to assign a Principal who does not have the PRINCIPAL role, the service throws a BadRequestException (HTTP 400).
- If a user attempts to access an endpoint outside their role's base path, Spring Security returns HTTP 403 Forbidden before the request reaches the controller.

2.2.2.2 Workflow-Enabled Club and Event Approval

Description and Priority: Club formation and event sanctioning require time-consuming, multisignature manual workflows. This feature digitizes the process, providing a sequential, transparent approval path.

Stimulus/Response Sequence: A Student initiates a Club request, which generates a digital workflow task. The task must be acted upon sequentially by the Professor/Guide, then the HOD, and finally the Principal. The Club is officially approved only upon the Principal's final sanction.

Functional Requirements:

- The Student must be able to submit a Club Creation Request specifying a guide teacher (by UUID), club name, description, and optional logo URL. Initial status is set to PENDING_HOD.
- The HOD must be able to view club requests filtered by status=PENDING_HOD and approve (advancing to PENDING_PRINCIPAL) or reject (setting REJECTED).
- The Principal must be able to view club requests filtered by status=PENDING_PRINCIPAL and make the final approve (APPROVED) or reject (REJECTED) decision.
- All state transitions must be validated server-side; invalid transitions (e.g., Principal approving a PENDING_HOD club) must return HTTP 400.
- Students must be able to join APPROVED clubs via POST /api/student/clubs/{clubId}/join.

Main Flow:

- Student calls POST /api/student/clubs with club details. ClubServiceImpl sets status to PENDING_HOD.
- HOD calls GET /api/hod/club-requests?status=PENDING_HOD to see pending requests for their department.
- HOD calls POST /api/hod/club-requests/{id}/approve. Status advances to PENDING_PRINCIPAL. An asynchronous email notification is sent to the student.
- Principal calls GET /api/principal/club-requests?status=PENDING_PRINCIPAL to see clubs awaiting final approval.
- Principal calls POST /api/principal/club-requests/{id}/approve. Status set to APPROVED. Student is notified via email.
- Club appears in GET /api/student/clubs with status APPROVED, and the isMember / isOwner flags allow the frontend to render the correct action button.

Exceptional Flow:

- If the HOD rejects the request (POST /api/hod/club-requests/{id}/reject), status is set to REJECTED and the workflow terminates. The Principal never receives the request.
- If the Principal attempts to approve a club that is still PENDING_HOD, ClubServiceImpl throws a BadRequestException (HTTP 400): "Club is not pending principal approval."

2.2.2.3 Three-Section Academic Progress Tracker

Description and Priority: Consolidating a student's performance in mandatory academic components (Project, Seminar, Internship) is often manual. This feature centralizes tracking, ensuring that no important information will be lost.

Stimulus/Response Sequence: A Student makes a submission. The Professor accesses the submission via the batch management dashboard and gives remarks (pass/fail/suggestions). This action instantly updates the student's personal progress dashboard.

Functional Requirements:

- The Professor must be able to create BatchSections of type PROJECT, SEMINAR, or INTERNSHIP under their batches, each with a title, description, and deadline date.
- The Student must be able to submit work against a specific BatchSection (by sectionId) via POST /api/student/submissions/{sectionId}, supporting both individual and team submissions with file URLs stored in Firebase Storage.
- The Professor must be able to view all submissions for a section via GET /api/professor/submissions/{sectionId} and give a remark and SubmissionStatus (UNDER_REVIEW, APPROVED, REJECTED) via PUT /api/professor/submissions/{id}/remark.
- The Student must be able to view their own submissions and professor remarks via GET /api/student/submissions.
- The Student's section selector on the Submissions page must be dynamically populated from GET /api/student/sections, returning only BatchSections relevant to their enrolled department.

Main Flow:

- Professor calls POST /api/professor/batches/{batchId}/sections with title, description, sectionType (PROJECT/SEMINAR/INTERNSHIP), and deadlineDate.
- Student accesses GET /api/student/sections to get the list of available sections for their department. The section selector is populated from this response.
- Student calls POST /api/student/submissions/{sectionId} with submissionType (INDIVIDUAL or TEAM), fileUrl, description, and for team submissions — teamName and teamMemberIds.
- Professor calls GET /api/professor/submissions/{sectionId} to review all submissions for that section.
- Professor calls PUT /api/professor/submissions/{id}/remark with a remark string and a SubmissionStatus. The reviewing professor's identity is automatically recorded in the reviewedBy field.
- Student's submission record instantly reflects the new status and remarks via GET /api/student/submissions.

Exceptional Flow:

- If a Professor attempts to save remarks without providing a remark string or status, Bean Validation (@Valid) rejects the request with HTTP 400 before reaching the service layer.

- For team submissions, remarks are applied to the team submission record which is linked to all teamMemberIds, ensuring all team members see the feedback on their individual dashboards.

2.2.3 Non-Functional Requirements

Table 2.1 : Non-Functional Requirements

Requirement	Description
Operating System	Linux/Windows Server for backend hosting; cross-platform web browser access for frontend
Backend Server	Spring Boot 4.0.3 embedded Tomcat, port 8080; Java 17
Frontend	React 18 + Vite build; served via static hosting or Node server
Database	PostgreSQL (campus_nexus); Spring Data JPA / Hibernate with ddl-auto: update
Authentication	Stateless JWT (jjwt 0.12.6), Spring Security 7; InvalidatedToken table for logout revocation
File Storage	Firebase Storage (firebase-admin 9.4.2); files accessed via public download URL
Push Notifications	Firebase Cloud Messaging (FCM) via firebase-admin SDK
Email	Spring Mail via SMTP (Gmail, port 587); @Async for non-blocking dispatch
Payments	Stripe Java SDK v31.1.0 (StripeClient API); webhook endpoint at /api/webhook/stripe
API Documentation	Springdoc OpenAPI / Swagger UI at /swagger-ui.html
Hardware	Standard desktop, laptop, or mobile device with internet connectivity

2.2.4 Deployment Environment

Table 2.2 : Deployment Environment

Component	Specification for CMS Platform
Operating System	Linux/Windows Server OS for hosting, accessible via standard web browsers (Cross-platform accessibility).
Servers Used	BaaS Platform (e.g., Firebase, AWS, Azure), Application Server (Web/API Server).

Server-side Database	Realtime Database (for dynamic content/IM) and/or Managed SQL/NoSQL (for structured academic records/hierarchy).
Client App Implementation	HTML5, JavaScript, CSS (Web-based application).
Hardware Platform	Standard desktop, laptop, or mobile device with internet connectivity.
Internet Connection	Required for all functioning and data synchronization.

2.2.5 External Interface Requirement

2.2.5.1 User Interfaces

- **Role-Specific Dashboards:** A unique dashboard is presented per role on login — Campus Admin sees system-wide statistics (total colleges, users, events, registrations, clubs, active students); Principal and HOD see department/college-scoped data; Professor sees batch, submission, and availability management; Student sees their events, clubs, timetable, progress, and submissions.
- **Workflow Status Indicators:** For club approvals, each ClubResponse carries a status field (PENDING_HOD, PENDING_PRINCIPAL, APPROVED, REJECTED) and a statusLabel string displayed on the relevant dashboards. The frontend filters by status query parameter to show only actionable items.
- **Academic Tracker View:** The Student's submissions page dynamically loads sections from GET /api/student/sections. Each SubmissionResponse shows the current status, professor remark, remarkAt timestamp, and reviewedByName.
- **Contextual Flags:** EventResponse includes isRegistered; ClubResponse includes isMember and isOwner — these flags are computed server-side and used by the frontend to render the correct action (Register / Already Registered, Join / Member / Owner) without additional API calls.

2.2.5.2 Software Interfaces

- **Spring Security 7 + JWT:** JwtAuthFilter intercepts all requests, validates Bearer tokens, checks the InvalidatedToken blacklist, and populates the SecurityContext with the user's role.
- **Spring Data JPA Repositories:** 17 JPA repositories handle all database reads and writes. Custom JPQL queries support filtered lookups (e.g., events by student scope, broadcasts by college/department, club requests by status).
- **Firebase Admin SDK:** FirebaseStorageService uploads files and returns public URLs. FCMService sends targeted push notifications via sendToToken and topic-based broadcasts via sendToTopic.

- **Stripe Java SDK v31:** StripeService uses the StripeClient API to create PaymentIntents. WebhookController verifies the Stripe-Signature header before processing payment_intent.succeeded events.
- **Spring Mail (@Async):** NotificationServiceImpl sends email confirmations and alerts on a separate thread, ensuring HTTP response times are unaffected by SMTP latency.

2.2.5.3 Communication Interfaces

All client-server communication is performed over HTTPS using RESTful JSON APIs. The Axios HTTP client on the frontend attaches JWT Bearer tokens via a request interceptor and handles 401 responses by automatically attempting token refresh via POST /api/auth/refresh. If the refresh fails, the user is logged out and redirected to the login page. For real-time feedback such as ticket confirmation, Firebase Cloud Messaging (FCM) is used to push notifications directly to the student's registered device token — stored via PUT /api/student/profile with an fcmToken field. Stripe communicates payment success asynchronously via a signed webhook POST to /api/webhook/stripe, verified using the Stripe-Signature header.

2.3 Project Process Modeling

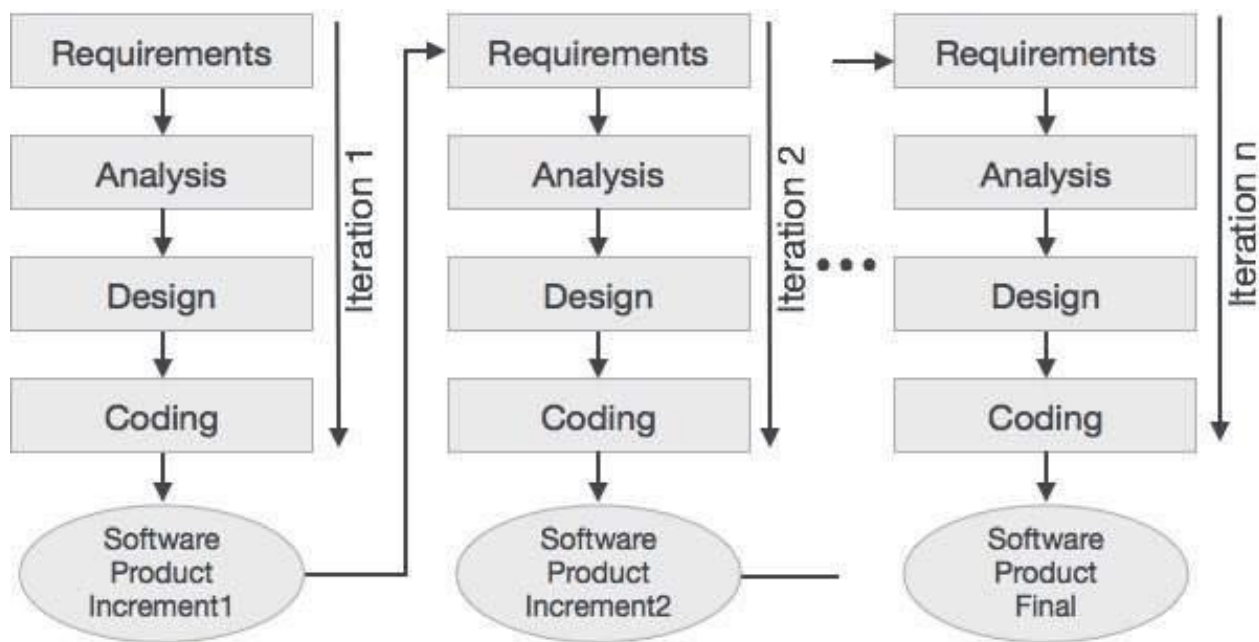


Figure 2.1 : Project Process Model (Scrum)

To develop the Campus Management System (CMS), the **Scrum Model**, an agile, iterative, and incremental process model for software development, is adopted.

Scrum combines the iterative philosophy of evolutionary approaches with a framework emphasizing flexibility, rapid deployment, and continuous feedback. Instead of delivering a single

final product, Scrum develops the CMS through a series of short, fixed-length cycles called **Sprints** (typically 2-4 weeks). Each Sprint produces a potentially shippable increment of the software.

Application of Scrum to CMS Development

The Scrum Model is particularly well-suited for this project due to the following characteristics:

- **Handling Evolving Requirements (Multi-Role Features):** The CMS features, such as the sequential Club/Event Approval workflow and the intricate Multi-Level Role Assignment, require constant refinement based on feedback from different stakeholders (Campus Admin, Principal, HOD). Scrum's iterative nature allows requirements to change and be prioritized with high flexibility, ensuring the product always aligns with the users' most pressing needs.
- **Incremental Delivery of Core Functionality:** The project will deliver operational increments early in the development timeline. The first increment (Sprint) can focus on the core product—basic user authentication, profile creation, and communication features. Subsequent Sprints will deliver the advanced, complex features incrementally, such as:
 - Increment 2: Multi-Level Role Assignment (Campus Admin/Principal features).
 - Increment 3: Academic Progress Tracker (Professor/Student submission and remarks).
 - Increment 4: Workflow-Enabled Approvals and Payment Management (Club/Event features).
- **Managing Technical Risks:** By delivering functionality in increments, the team can address technical challenges, such as integrating the complex RBAC logic or the real-time data synchronization necessary for academic tracking, in early Sprints, reducing overall project risk.
- **Continuous Stakeholder Review:** The Scrum process includes frequent Sprint Reviews, where the developed increment is demonstrated to stakeholders (teachers, students, admin), providing a platform for continuous evaluation by the user. This feedback loop is vital for ensuring that features like the Three-Section Academic Progress Tracker or the specific TimeBased Teacher Availability Marking are implemented correctly and intuitively.

This process ensures that the CMS is built upon a solid, validated foundation, delivering demonstrable value to the campus community with each cycle.

2.4 Cost & Efforts Estimates

The Cost of the Campus Management System (CMS) project is determined by the cost of the necessary hardware (mainly CPU/Cloud services) and the cost of the human effort put into development. The Basic COCOMO (Constructive Cost Model) is used to estimate the project's effort, duration, and staffing requirements.

1. COCOMO Model Parameters

The following parameters and estimates are applied to the model:

- **Project Class:** The CMS project is determined to fit the characteristics of the **Semi-Detached Mode**. This classification typically applies to projects where requirements are somewhat rigid and the development team has mixed experience.
- **Estimated Size (KLOC):** The estimated size of the project is **10,000 Delivered Source Instructions (LOC)**, which corresponds to **KLOC = 10** (Kilo lines of code).
- COCOMO Coefficients for Semi-Detached Mode:

Table 2.3 : COCOMO Coefficient

Coefficient	Value
a _b	3.0
b _b	1.12
c _b	2.5
d _b	0.35

2. Basic COCOMO Equations

The calculations are based on the following standard COCOMO equations:

Effort Applied (E) = $a_b(\text{KLOC})^{b_b}$

Development Time (D) = $c_b(E)^{d_b}$

People Required (P) = E/D

3. Calculations and Results

Applying the parameters for the Semi-Detached Mode with KLOC=10:

Table 2.4 : Calculation and Results

Metric	Formula	Calculation	Result
Effort Applied (E)	$E = 3.0 * (10)^{1.12}$ [man-months]	$E = 3.0 * 13.18$	E = 39.54 man-months
Development Time (D)	$D = 2.5 * (39.54)^{0.35}$ [months]	$D = 2.5 * 3.62$	D = 9.055 months
People Required (P)	$P = E / D$	$P = 39.54 / 9.055$	P = 4.36 people

The Basic COCOMO model estimates that the Campus Management System project will require 39.54 man-months of effort, take approximately 9.055 consecutive months to develop, and necessitate a project team size of around 4 to 5 people.

2.5 Project Scheduling

2.5.1 Time Line Chart



Figure 2.2 : Time Line Chart

Chapter 3

ANALYSIS & DESIGN

3.1 Introduction

This chapter covers the analysis and design of the considered system, the Campus Management System (CMS). Analysis begins with the IDEA Matrix to evaluate the strategic impact of the project's unique features, followed by the formal Mathematical Model and feasibility study. The design phase details the architectural blueprint and structural components using various UML diagrams to illustrate the complex multi-level roles, sequential workflows, and real-time academic tracking mechanisms of the system.

3.2 IDEA Matrix

The IDEA matrix evaluates the impact of the key innovations and goals of the Campus Management System (CMS) across four dimensions: I (Innovate/Improve/Increase), D (Decrease/Detect/Drive), E (Exchange/Enhance/Extract), and A (Avoid/Assure/Associate).

I Elements (Innovate, Improve, Increase)

Table 3.1 : I Element IDEA Matrix

Use	Parameters Affected	Description
Improve	Administrative Efficiency.	Improves efficiency of role assignment, college creation, and event/club approvals by digitizing multi-stage workflows through 88 dedicated REST endpoints protected by Spring Security 7.
Increase	Transparency, Data Consistency.	Increases transparency for students (isRegistered flag per event, isMember/isOwner per club, real-time submission remarks) and for administrators monitoring dashboard statistics across colleges.
Innovate	Core Functionality, Hierarchy Management.	Introduces a five-tier RBAC hierarchy (CAMPUS_ADMIN → PRINCIPAL → HOD → PROFESSOR → STUDENT) with JWT-based stateless authentication, token revocation via InvalidatedToken blacklist, and context auto-injection in controllers.

D Elements (Decrease, Detect, Drive)

Table 3.2 : D Element IDEA Matrix

Use	Parameters Affected	Description
Detect	Workflow Status.	The system automatically detects the current ClubStatus (PENDING_HOD, PENDING_PRINCIPAL, APPROVED, REJECTED) and EventStatus (UPCOMING, ONGOING, COMPLETED, CANCELLED) to enforce valid state transitions server-side.
Decrease	Administrative Overheads, Paperwork.	Decreases manual approvals by enforcing digital workflows; eliminates manual payment collection through Stripe PaymentIntent integration with automatic ticket confirmation via webhook.
Drive	Functional Compliance. Ability,	Driven by Spring Security 7 URL-pattern role rules ensuring each role accesses only their designated base path (/api/admin, /api/principal, /api/hod, /api/professor, /api/student).

E Elements (Exchange, Enhance, Extract)

Table 3.3 : E Element IDEA Matrix

Use	Parameters Affected	Description
Exchange	Information, Communication.	Exchanges broadcast messages at CAMPUS, COLLEGE, or DEPARTMENT level with scoped filtering; professor notes and submission files are exchanged via Firebase Storage public URLs.
Enhance	Academic Monitoring, User Control.	Academic capabilities enhanced by the Three-Section Progress Tracker (PROJECT, SEMINAR, INTERNSHIP) with SubmissionStatus lifecycle and professor remarks recorded with timestamp and reviewer identity.
Extract	Financial Data, Event Registration.	Event registration and payment data extracted and retained via EventRegistration records linked to TicketStatus (PENDING, CONFIRMED, CANCELLED) and PaymentStatus (PENDING, SUCCESS, FAILED).

A Elements (Avoid, Assure, Associate)

Table 3.4 : A Element IDEA Matrix

Use	Parameters Affected	Description
Avoid	Manual Delays.	Avoids manual delays in club and event approval by enforcing sequential digital workflows with status-filtered endpoints (?status=PENDING_HOD, ?status=PENDING_PRINCIPAL).
Assurance	Data Integrity.	Assures no data is lost during complex transactions — JWT logout revokes tokens immediately via InvalidatedToken table; Stripe webhook confirms tickets only after payment_intent.succeeded verification.
Associate	Adaptability, Ecosystem Integration.	All APIs return standardized ApiResponse<T> wrappers ensuring a consistent contract between the Spring Boot backend and the React frontend; third-party integrations (Stripe, Firebase, Spring Mail) are injected as Spring-managed beans.

3.3Mathematical Model

Let S be the proposed **Campus Management System (CMS)**. The system is defined by its components: Input, Functions, Output, Success conditions, and Failure conditions.

$$S = \{\text{Input, Functions, Output, Success, Failure}\}$$

A. Input

The input set $\text{Input} = \{I_1, I_2, I_3, I_4, I_5\}$ represents the primary data or commands received by the system:

- I_1 : Hierarchy Change Data (College name, Principal ID, HOD ID).
- I_2 : Workflow Request Data (Club request, Event details, Approval decision).
- I_3 : Academic Submission Data (Project file, Seminar file, Internship file).
- I_4 : Academic Feedback Data (Professor's remarks, status update).
- I_5 : Payment Data (Ticket purchase information).

B. Functions

The function set $\text{Functions} = \{F_1, F_2, F_3, F_4, F_5\}$ represents the core logic applied to the input data:

- F_1 : Function for Role Assignment (Assigning Principal/HOD roles).
- F_2 : Function for Sequential Workflow Routing and Status Update (Managing club/event approval).

- F_3 : Function for Submission Handling and Remark Storage (Saving academic files and Professor feedback).
- F_4 : Function for Payment Processing and Ticket Generation.
- F_5 : Function for Time-Based Availability Marking.

C. Output

The output set $\text{Output} = O_1, O_2, O_3, O_4, O_5$ represents the results of the functions:

- O_1 : Role Updated/Hierarchy Established.
- O_2 : Workflow Status Displayed (e.g., Pending HOD, Approved).
- O_3 : Academic Progress Updated (Remarks and status visible on the student dashboard).
- O_4 : Event Ticket Generated (After payment success).
- O_5 : Broadcast Message Sent.

D. Success

The success set $\text{Success} = S_1, S_2, S_3, S_4$ describes desired operational outcomes:

- S_1 : Hierarchy Success: Successfully established a valid college-department-role hierarchy, and the user's permissions are updated instantly.
- S_2 : Workflow Success: Club/Event request is routed, processed, and accepted/denied according to the defined sequential path.
- S_3 : Academic Success: Student submission is successfully stored, and the Professor's remarks are accurately logged against the student/team record.
- S_4 : Transaction Success: Payment is successfully processed, and the event ticket is correctly generated and issued.

E. Failure

The failure set $\text{Failure} = F_1, F_2, F_3$ describes critical system failures:

- F_1 : Hierarchy Conflict: Attempted role assignment fails due to non-existent user or invalid college ID.
- F_2 : Workflow Interruption: Sequential workflow approval is stuck or bypassed incorrectly.
- F_3 : Data Inconsistency: Professor remarks are logged but fail to display correctly on the student's progress dashboard.
- F_3 : Payment Failure: Payment is received by the gateway but the ticket fails to generate.

3.4 Feasibility Analysis

The system uses PostgreSQL with Spring Data JPA. Key query operations include: scoped event retrieval for students (filtered by college + department), club request filtering by ClubStatus and department/college scope, and broadcast filtering by BroadcastLevel. All queries are backed by indexed foreign keys on college_id, department_id, and status columns, ensuring $O(\log n)$ lookup performance at scale. The isRegistered and isMember flags are computed in-memory during response mapping in EventServiceImpl and ClubServiceImpl respectively, avoiding additional round-trips to the database.

3.5Architecture Diagram

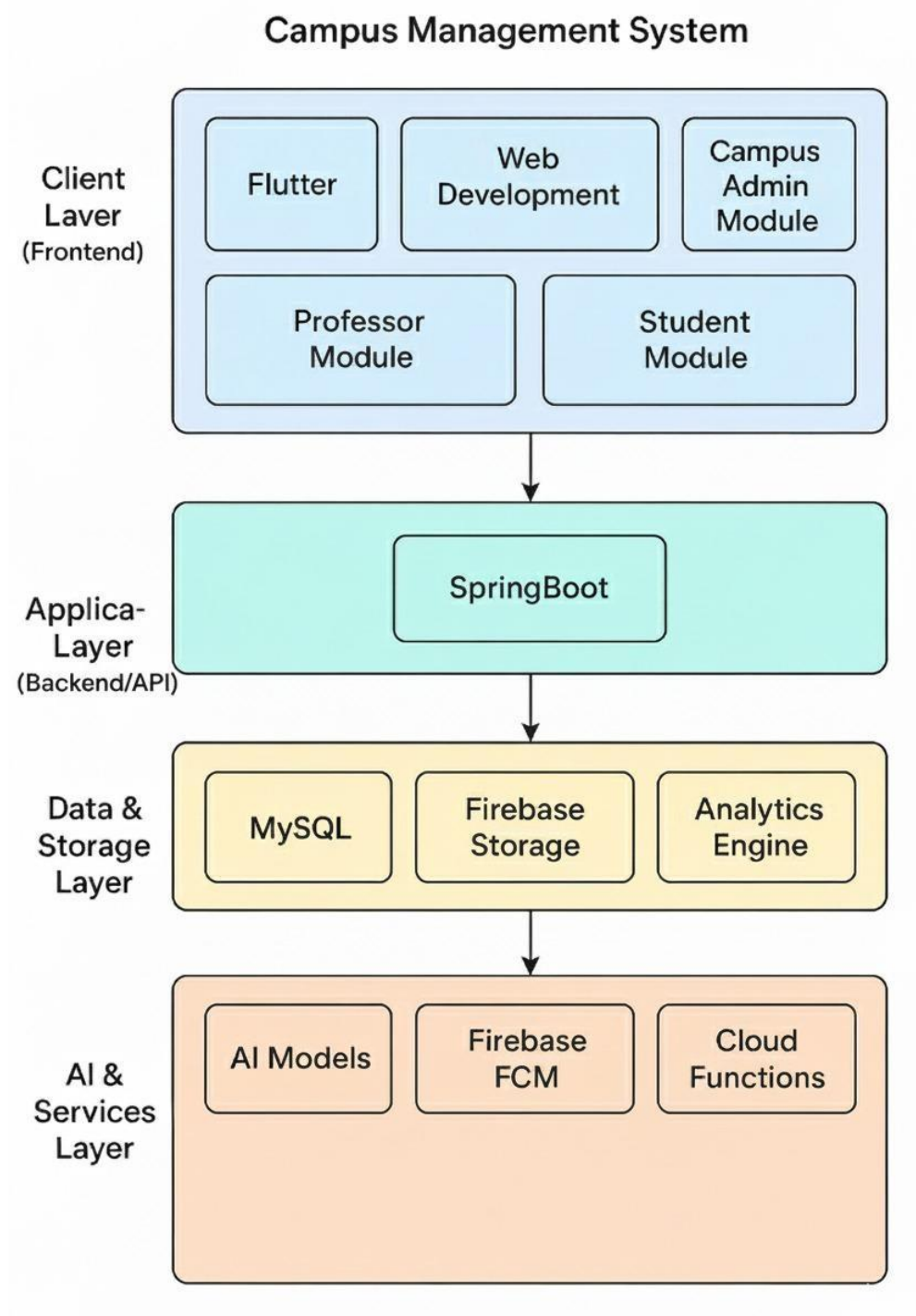


Figure 3.1 : System-Architecture Diagram

- The System Architecture Diagram (Figure 3.1) illustrates the layered structure of the Campus Management System, showcasing the interaction between its frontend, backend,

and data layers. The architecture is divided into four main layers — Client Layer, Application Layer, Data & Storage Layer, and AI & Services Layer.

- The Client Layer (Frontend) includes the modules developed using Flutter and Web Development, such as the Campus Admin Module, Professor Module, and Student Module, providing user interaction interfaces.
- The Application Layer (Backend/API) is built using Spring Boot, which handles business logic, communication between modules, and integration with the data layer.
- The Data & Storage Layer manages data persistence and analytics through MySQL, Firebase Storage, and an Analytics Engine, ensuring secure and efficient data handling.

Finally, the AI & Services Layer supports intelligent features and notifications using AI Models, Firebase Cloud Messaging (FCM), and Cloud Functions, enhancing the overall system performance and automation capabilities.

3.6UML Diagrams

3.6.1 Use-Case Diagrams

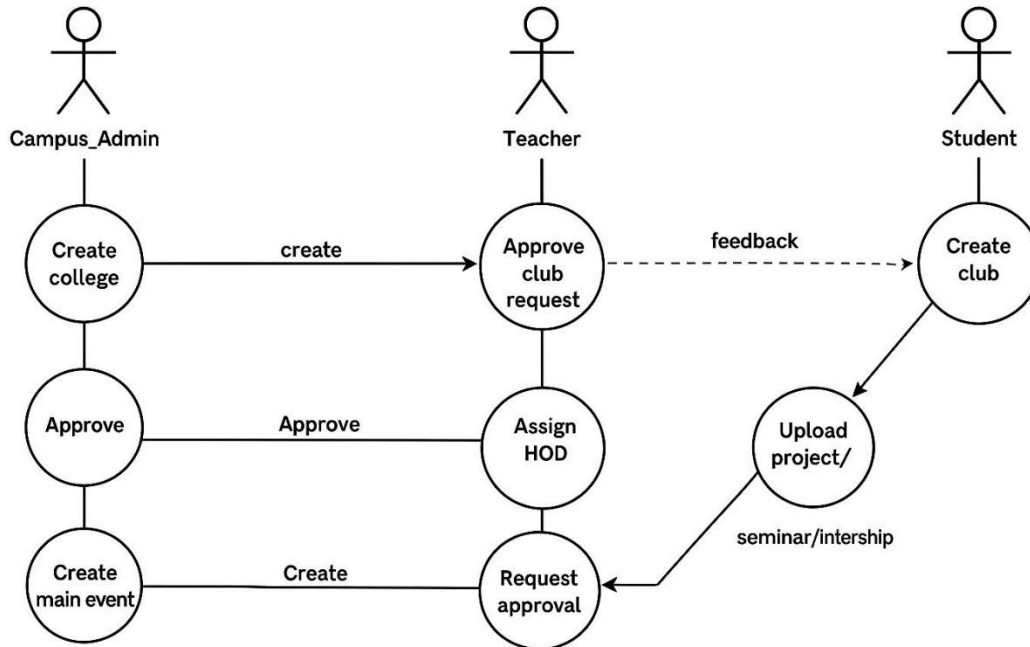


Figure 3.2 : Use- Case Diagram

The Use Case Diagram (Figure 3.2) represents the functional requirements of the Campus Management System. It involves three main actors — Campus Admin, Teacher, and Student — and illustrates their interactions within the system. As shown in Figure 3.2, the Campus Admin can create colleges, approve requests, and organize main events. The Teacher is responsible for approving club requests, assigning HODs, and sending approval requests. Meanwhile, the Student can create clubs, upload projects, seminars, and internships, as well as send feedback to teachers.

3.6.2 Activity Diagram

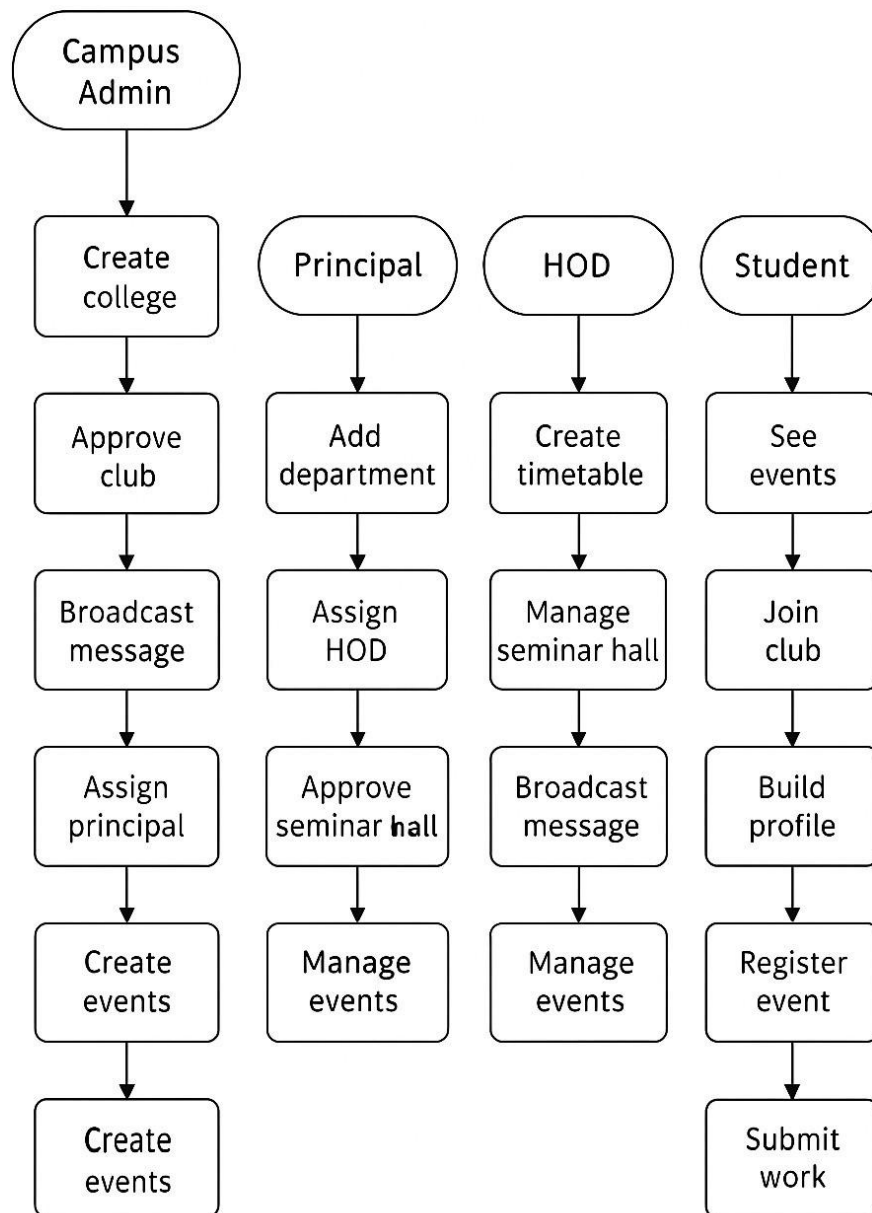


Figure 3.2 : Activity Diagram

The Activity Diagram (Figure 3.2) represents the flow of activities and interactions between different users in the Campus Management System. It highlights the key operations performed by the four main actors — Campus Admin, Principal, HOD, and Student.

The Campus Admin initiates the process by creating colleges, approving clubs, broadcasting messages, assigning principals, and managing events. The Principal adds departments, assigns HODs, approves seminar halls, and oversees event management. The HOD creates timetables, manages seminar halls, broadcasts messages, and handles departmental events. Meanwhile, the Student interacts with the system by viewing events, joining clubs, building profiles, registering for events, and submitting their work.

3.6.3 Class Diagrams

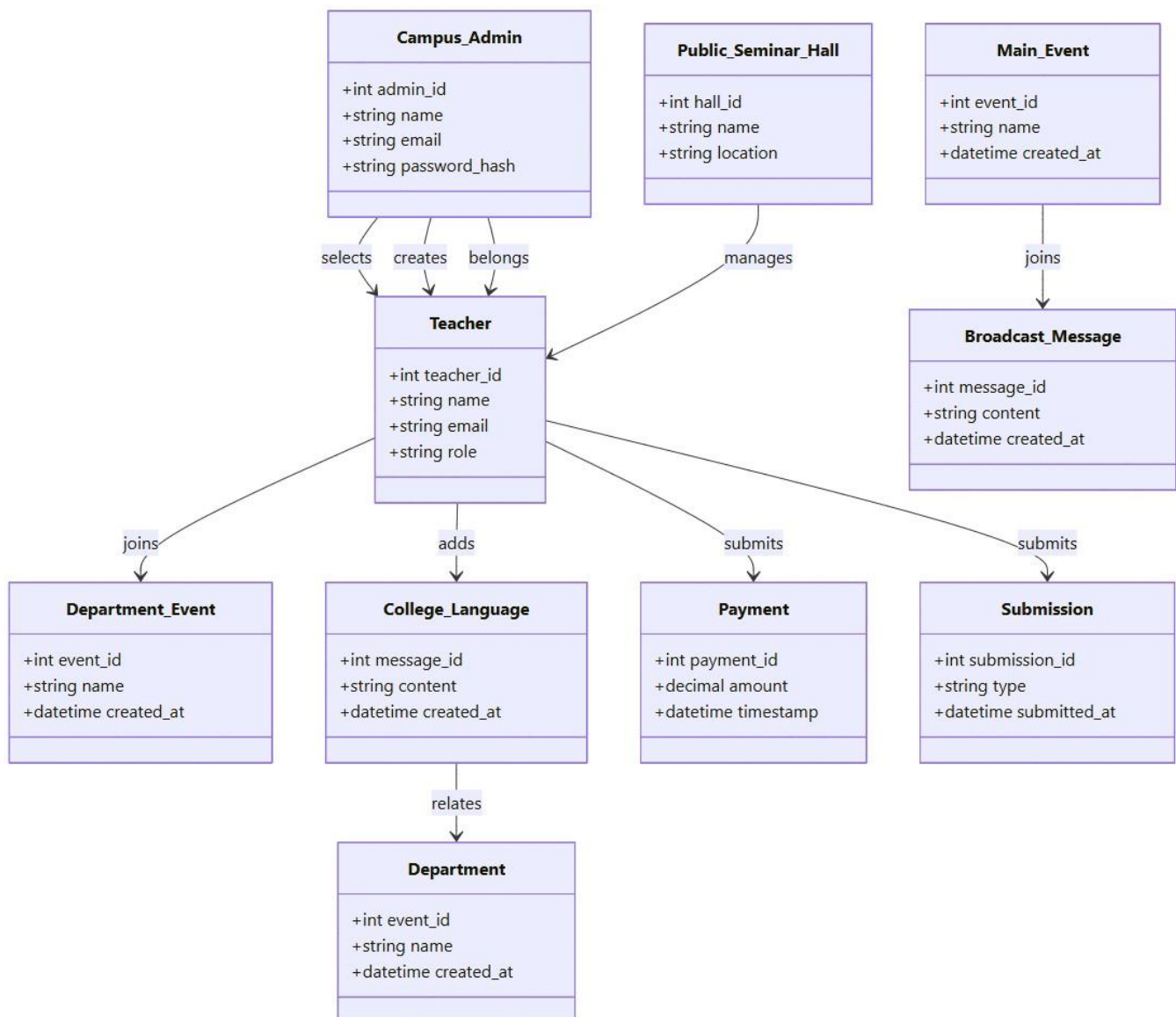


Figure 3.4 : Class Diagram

Class Responsibilities

1. Campus_Admin ○ Manages overall campus operations. ○ Creates colleges, assigns teachers, and manages authentication details like email and password.
2. Teacher ○ Acts as a key entity connecting admin, students, and departments. ○ Approves or manages events, submissions, payments, and messages.
 - Has attributes like teacher ID, name, email, and role (e.g., HOD, faculty).
3. Public_Seminar_Hall
 - Stores information about available seminar halls including ID, name, and location.
 - Managed by teachers for conducting public seminars and events.
4. Main_Event ○ Represents major campus events. ○ Linked with teachers and messages for event communication and participation.
5. Department_Event ○ Handles events specific to departments.
 - Joined and coordinated by teachers for departmental-level activities.
6. College_Language ○ Manages official communications or announcements within the college.
 - Contains message content and creation details, and relates to departments.
7. Department ○ Represents various departments within the college.
 - Linked with departmental events and official communications.
8. Broadcast_Message ○ Used for broadcasting announcements or updates to multiple users. ○ Created by teachers and associated with main events.
9. Payment ○ Handles all monetary transactions related to events or seminars.
 - Maintains details like payment amount and timestamp.
10. Submission
 - Manages submission of projects, reports, or internship details by student or teachers.
 - Stores submission type and date of submission.

3.6.4 ER Diagrams

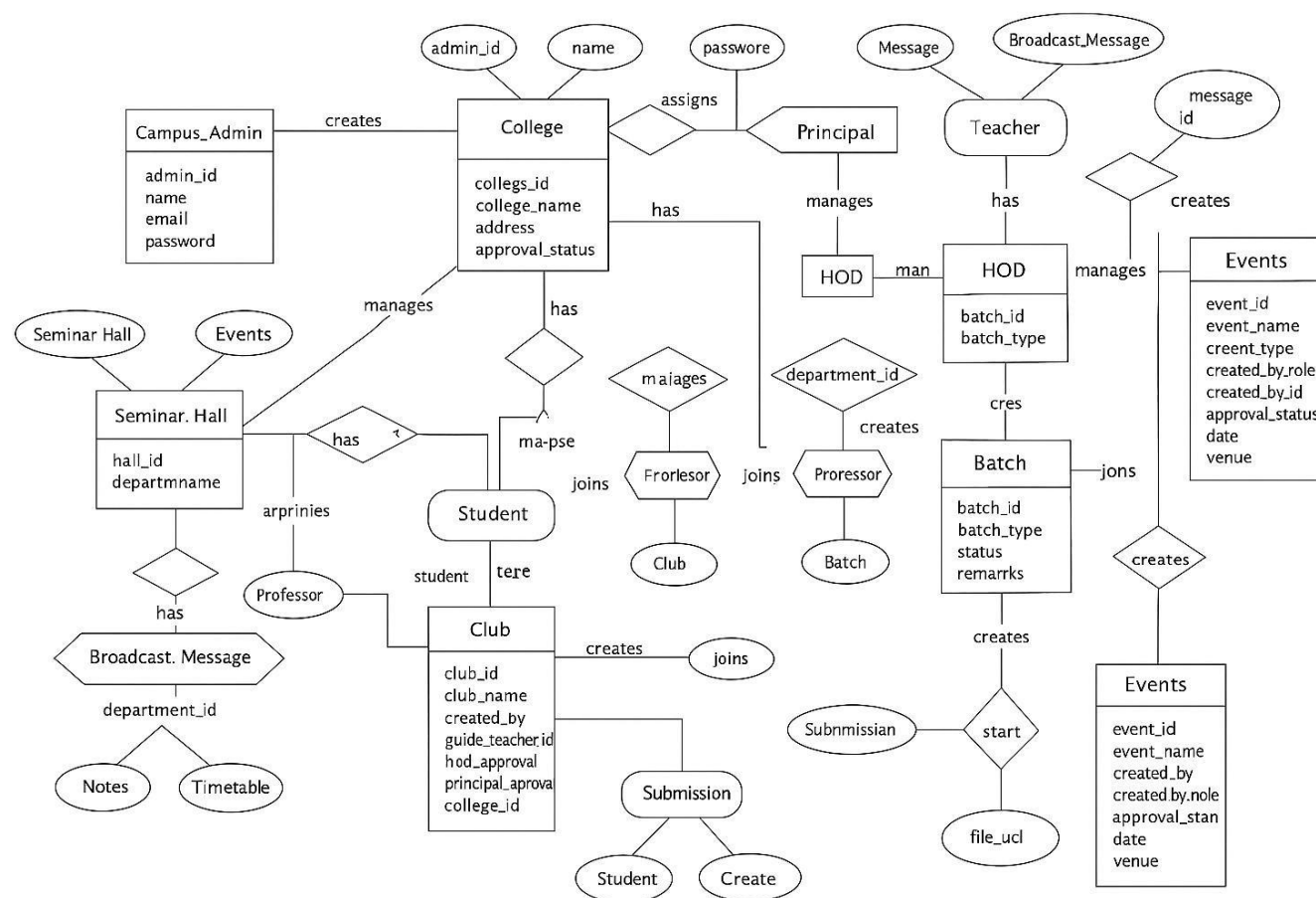


Figure 3.5 : ER Diagram

The Entity-Relationship (ER) Diagram (Figure 3.5) illustrates the database design of the Campus Management System, showing how different entities interact to manage campus activities efficiently. As shown in Figure 3.5, the system consists of key entities such as Campus_Admin, College, Principal, HOD, Teacher, Student, Club, and Event, each connected through specific relationships representing real-world interactions.

The Campus_Admin creates and manages colleges, while each College is associated with a Principal responsible for assigning HODs and overseeing departments. The HOD manages batches, seminar halls, and departmental events, whereas Teachers handle broadcasts, messages, and student submissions. Students join clubs, register for events, and submit their work under teacher or department guidance.

As shown in Figure 3.5, additional entities like Seminar_Hall, Submission, Broadcast_Message, Notes, and Timetable further support academic and administrative functions, ensuring smooth coordination between users and modules. Overall, this diagram provides a clear and systematic view of how data is organized and relationships are maintained within the system database.

3.6.5 Sequence Diagram

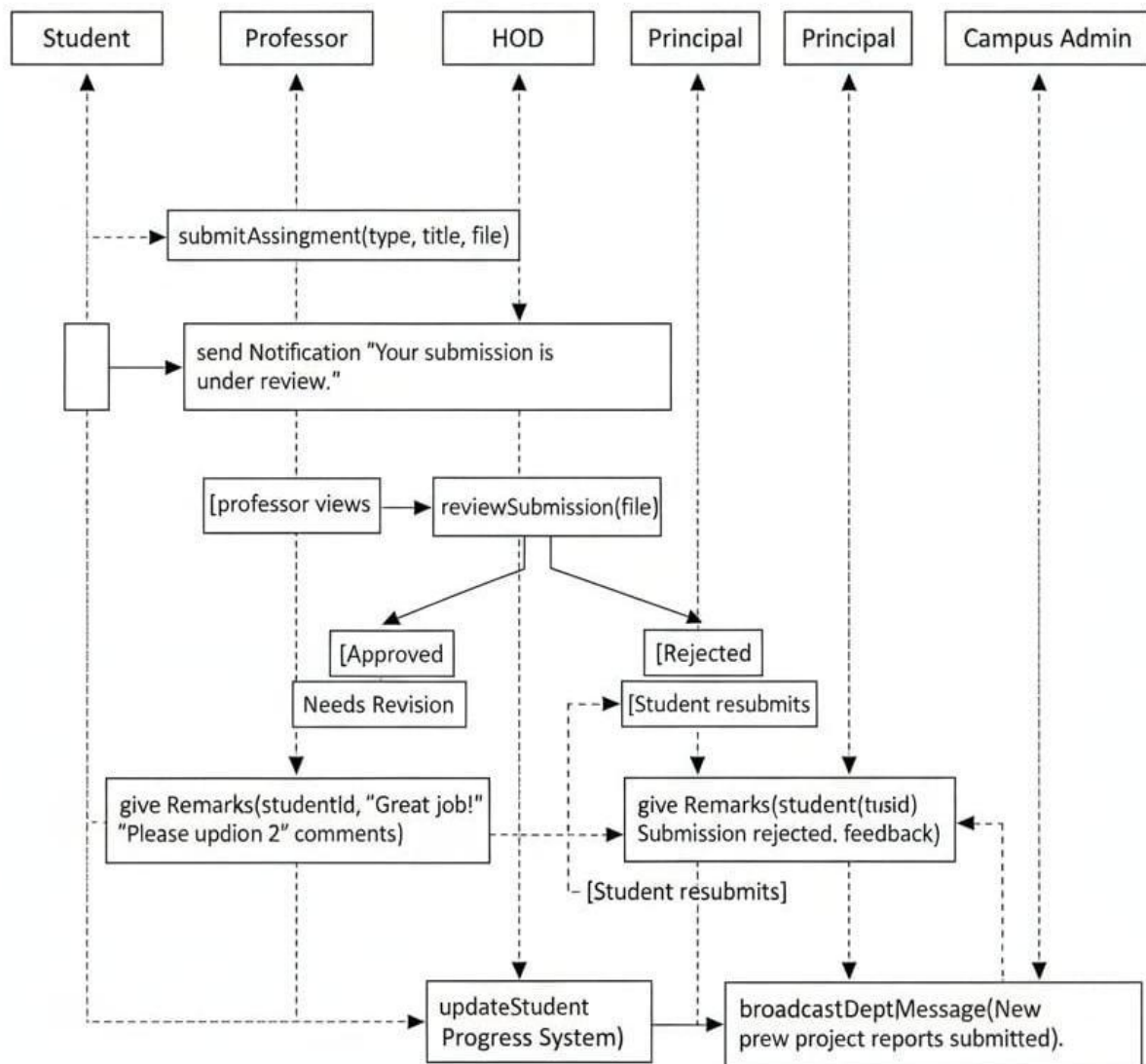


Figure 3.6 : Sequence Diagram Teacher(Professor Role : Student Submission and Remark)

The Sequence Diagram (Figure 3.6) illustrates the interaction between the Student, Professor, HOD, Principal, and Campus Admin during the process of student submission and evaluation. As shown in Figure 3.6, the sequence begins when a Student submits an assignment or project along with its type, title, and file. Upon submission, the system sends a notification stating, "Your submission is under review."

The Professor then reviews the submission and provides remarks based on its quality. If the submission is approved, remarks such as "Great job!" are recorded, whereas if it requires

revision or is rejected, the student is asked to make the necessary corrections and resubmit the work. As shown in Figure 3.6, once the submission is finalized, the system updates the Student Progress System and triggers a broadcast message to inform the department about new submissions or reports.

This diagram effectively demonstrates the communication flow and decision-making process involved in the submission-review cycle between students and faculty members within the Campus Management System.

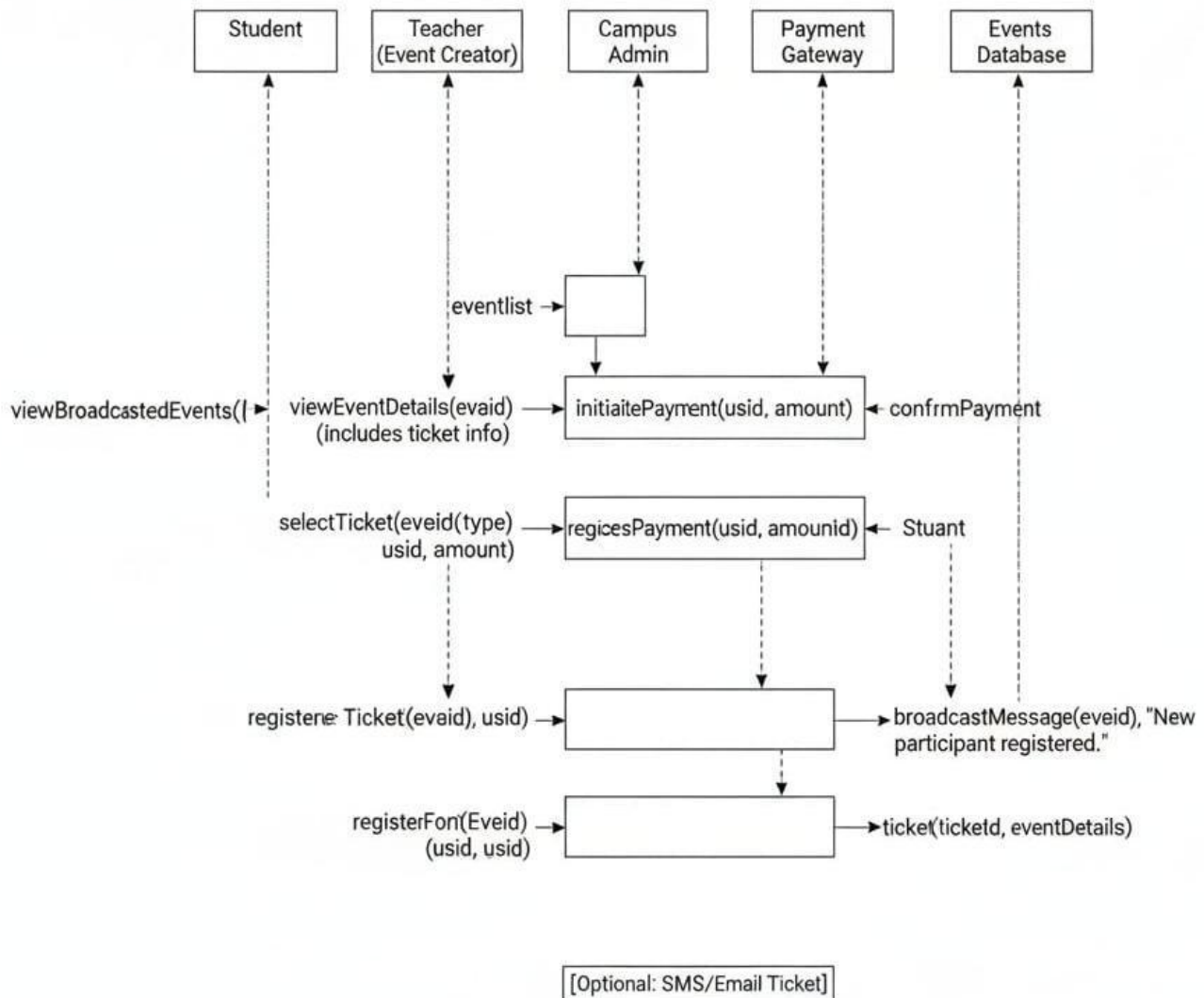


Figure 3.7 : Sequence Diagram Student: View Events & Register with Payment

The Sequence Diagram (Figure 3.7) demonstrates the process of how a Student views available events and completes registration through an integrated payment gateway in the Campus Management System. As shown in Figure 3.7, the interaction begins when the student accesses the list of broadcasted events and selects a specific event to view detailed information, including ticket availability and pricing.

Once the event is selected, the student initiates the payment process by sending the required details such as user ID and amount to the Campus Admin, which in turn communicates with the Payment Gateway to confirm the transaction. After successful payment, the system registers the student for the event, generates a ticket ID with corresponding details, and updates the Events Database.

As shown in Figure 3.7, a broadcast message is then sent to notify the event creator that a new participant has been registered. Optionally, the system can also send a confirmation ticket via

SMS or Email. This sequence efficiently captures the end-to-end flow of event registration and payment confirmation within the system.

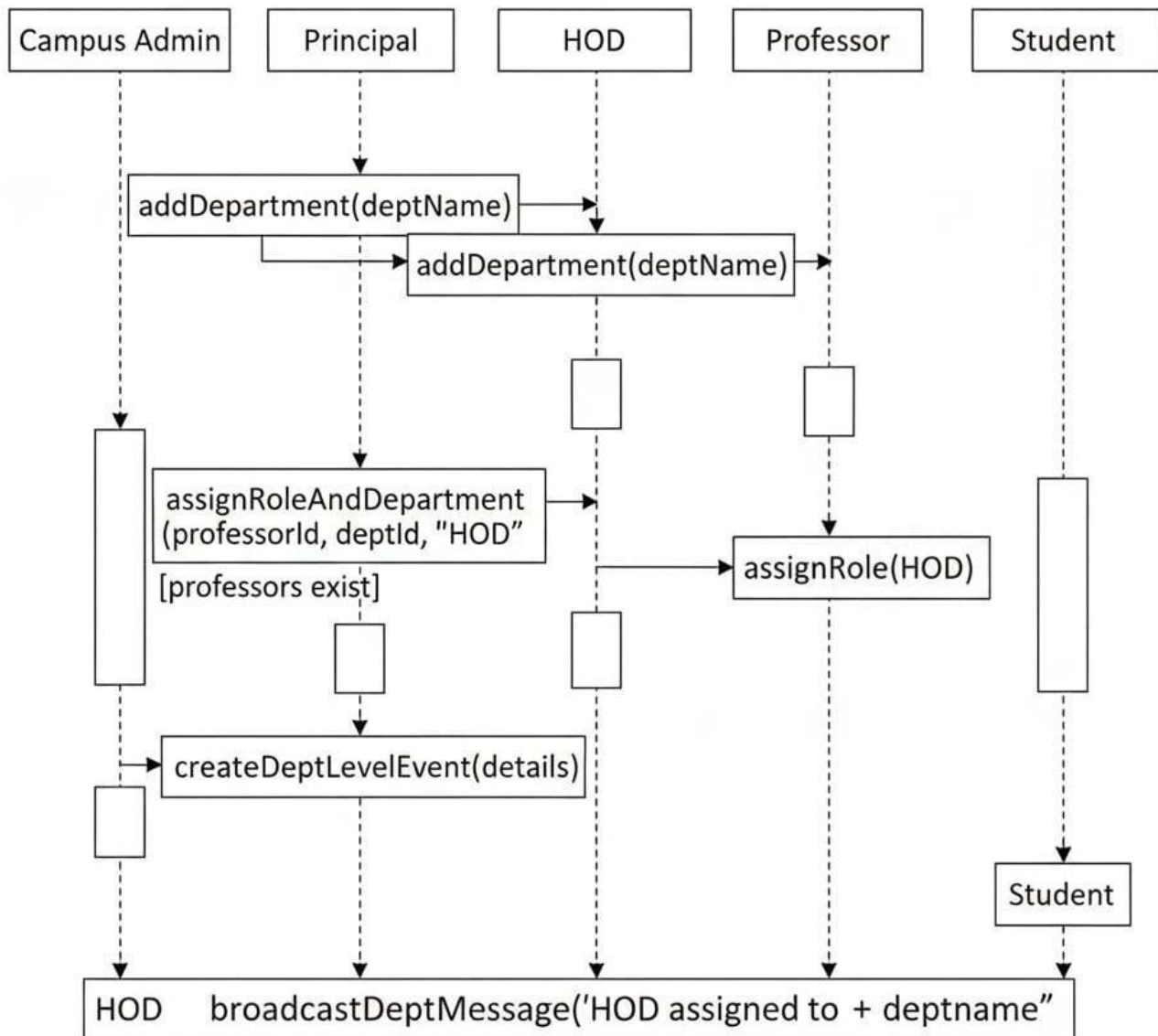


Figure 3.8 : Sequence Diagram Teacher(Principal Role: Add Department & Assign HOD

The Sequence Diagram (Figure 3.8) depicts the interaction between the Campus Admin, Principal, HOD, Professor, and Student during the process of adding a department and assigning an HOD in the Campus Management System. As shown in Figure 3.8, the process begins when the Campus Admin or Principal adds a new department by invoking the

`addDepartment(deptName)` function. Once the department is successfully created, the system proceeds to assign a Professor to the newly created department as an HOD.

The Principal executes the `assignRoleAndDepartment()` operation, linking the professor to the department with the role of HOD. If eligible professors already exist, the system automatically updates their assigned role using the `assignRole(HOD)` function. The HOD then creates department-level events and broadcasts a message to notify other users that a new department head has been appointed.

As shown in Figure 3.8, the entire sequence ensures proper hierarchical communication, where each role—Campus Admin, Principal, and HOD—collaborates to maintain departmental structure and notify all stakeholders through broadcast messages.

3.6.6 Component/Interface diagram

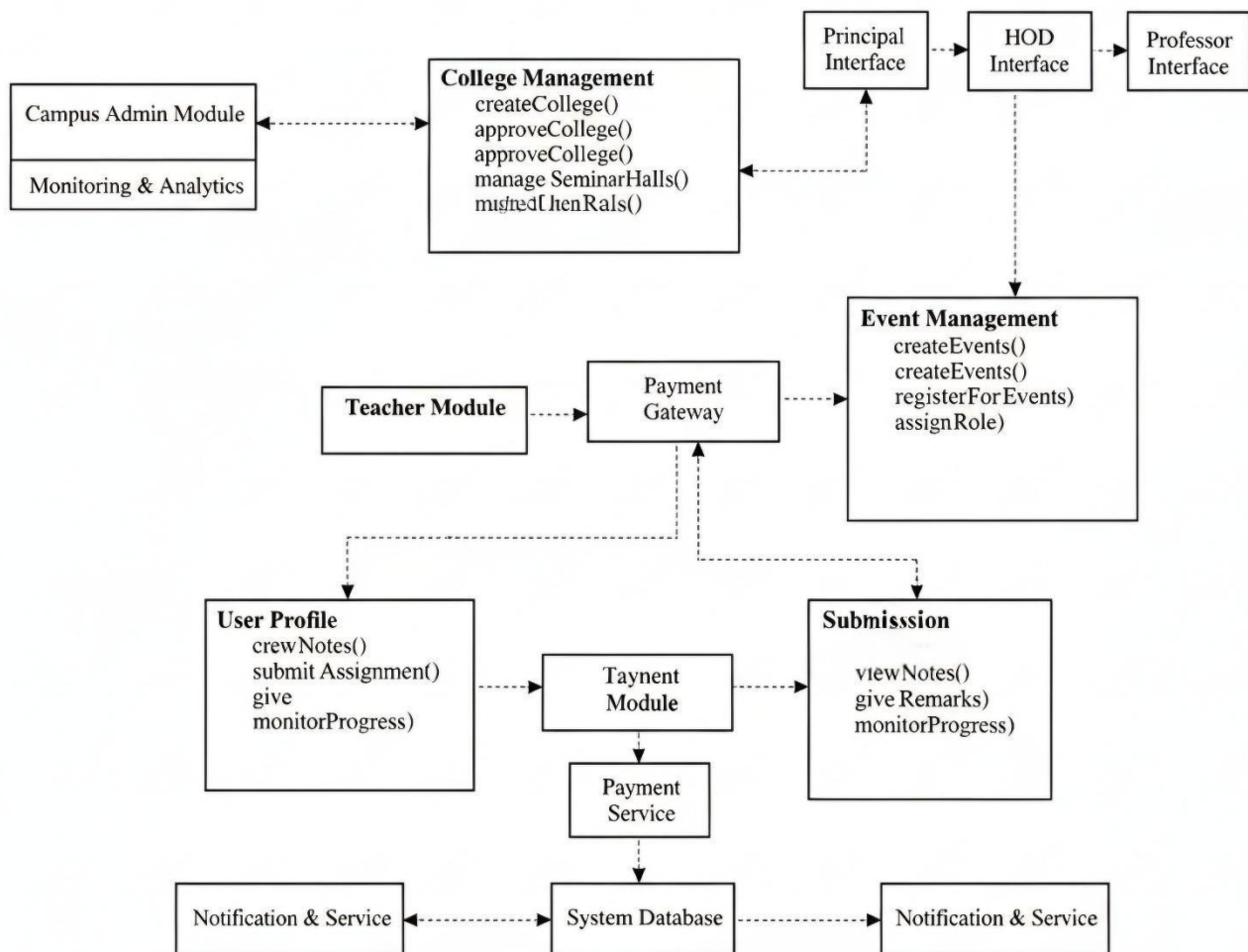


Figure 3.9 : Interface Diagram

The Interface Diagram (Figure 3.9) illustrates the interaction between various system modules and interfaces within the Campus Management System. As shown in Figure 3.9, the architecture connects multiple functional modules such as College Management, Event Management, Teacher Module, User Profile, and Submission, along with support components like Payment Gateway, System Database, and Notification & Service.

The Campus Admin Module oversees overall campus operations and analytics, while the College Management module handles operations like creating, approving, and managing colleges and seminar halls. The Event Management interface allows users such as principals,

HODs, and professors to create and manage events and assign roles. Meanwhile, the User Profile and Submission modules manage student interactions such as viewing notes, submitting assignments, and monitoring academic progress.

As shown in Figure 3.9, the Payment Gateway and Payment Module facilitate secure transactions related to event registration and other payments, while the Notification & Service component ensures timely communication and system updates. Overall, this interface diagram effectively demonstrates how each module collaborates to provide a seamless and integrated user experience across different roles within the system.

3.6.7 Deployment Diagrams

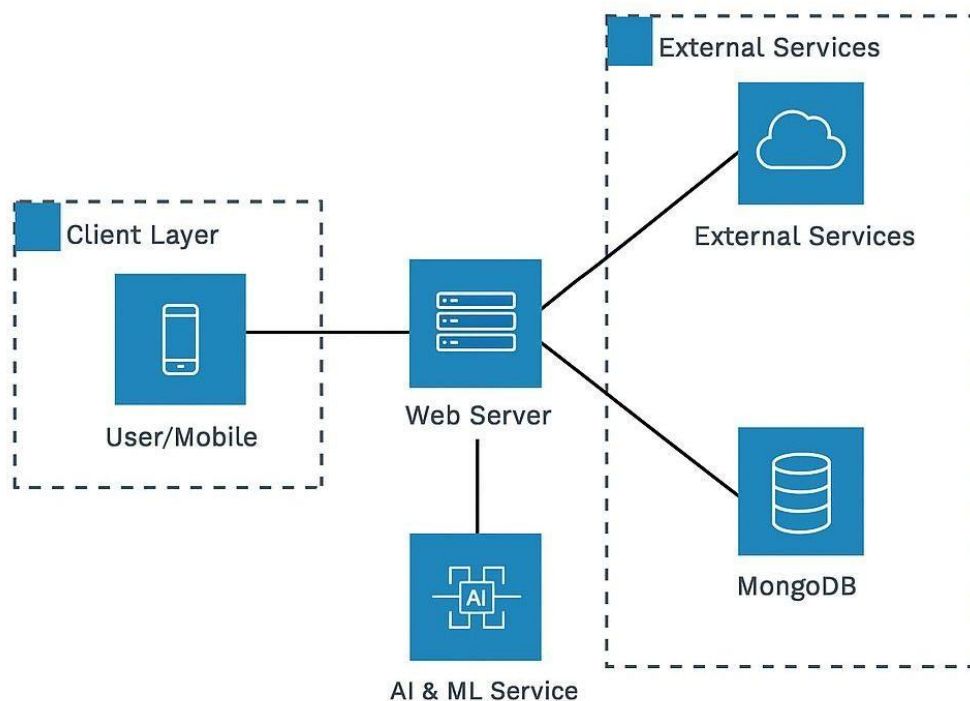


Figure 3.10 : Deployment Diagram

The Deployment Diagram (Figure 3.10) represents the physical architecture of the Campus Management System, showing how various components are deployed and interact during execution. As shown in Figure 3.10, the system is structured into multiple layers — the Client Layer, Web Server, Database, AI & ML Service, and External Services — all working together to ensure seamless communication and processing.

The Client Layer consists of users accessing the system through mobile or web interfaces, which communicate directly with the Web Server. The Web Server acts as the central node, handling user requests, interacting with the MongoDB database for data storage, and

connecting to AI & ML Services for intelligent processing tasks such as analytics or recommendations.

As shown in Figure 3.10, the External Services component provides third-party integrations such as authentication, cloud storage, or notification APIs that enhance system functionality. This deployment setup ensures scalability, modularity, and high performance, making the system reliable for both administrative and academic operations.

Chapter 4

IMPLEMENTATION AND CODING

4.1 Introduction

The implementation phase of CampusNexus: A Role-Based Multi-College Campus Management System focuses on converting the detailed system design into fully functional software modules. This chapter outlines the development approach, technologies employed, and key coding structures that bring the system's role-driven campus administration capabilities to life. Special attention is given to realizing the Authentication and Authorization module, the College and Department Management module, the Event Management module with integrated Stripe payments, the Club Governance module, and the Student Academic module—which collectively form the system's core deliverables. The backend REST APIs, React-based frontend integration, and security layer implementation are described to provide a comprehensive view of the implementation.

The backend is implemented using Spring Boot 4.0.3 with Java 17, following a strict layered architecture: Controller → Service Interface → Service Implementation → Repository → PostgreSQL. Each module is encapsulated in its own controller class and backed by JPAManaged entities. The frontend is built with React 18 and Vite, with role-specific page directories under `src/pages/`, centralized API modules under `src/api/`, and Zustand-managed global state. All communication between frontend and backend is performed through 88 RESTful endpoints protected by JWT-based Spring Security 7.

4.2 Module Implementation

4.2.1 Operational Details

1. CampusNexus enforces a five-tier Role-Based Access Control (RBAC) hierarchy—CAMPUS_ADMIN, PRINCIPAL, HOD, PROFESSOR, and STUDENT—where every API endpoint is protected by Spring Security 7 and JWT bearer tokens, ensuring that each role can only access its designated set of 88 REST endpoints.
2. User authentication is handled through stateless JWT access tokens (24-hour expiry) and refresh tokens (7-day expiry). On logout, the token's JTI (JWT ID) is recorded in the InvalidatedToken table and checked by JwtAuthFilter on every subsequent request, effectively revoking stolen tokens before natural expiry.
3. The College Lifecycle Management module allows the CAMPUS_ADMIN to create colleges, autogenerate unique college codes, assign Principals, and toggle college status (PENDING → ACTIVE → INACTIVE) through dedicated REST endpoints under `/api/admin/colleges`.
4. The Event Management module supports multi-level events (CAMPUS, COLLEGE, DEPARTMENT, CLUB) with sub-event nesting, participant tracking, and a configurable status lifecycle (UPCOMING → ONGOING → COMPLETED / CANCELLED). Paid events trigger a Stripe PaymentIntent creation; the client secret is returned to the frontend to complete payment via Stripe.js.
5. A Stripe webhook endpoint (`/api/webhook/stripe`) listens for `payment_intent.succeeded` events. On receipt, the corresponding EventRegistration ticket status is updated from PENDING to

CONFIRMED and a Firebase Cloud Messaging (FCM) push notification is dispatched to the student's registered device token.

6. The Club Governance module implements a two-stage approval finite-state machine: a student submits a club request (PENDING_HOD); the HOD approves or rejects it; if approved the status advances to PENDING_PRINCIPAL; the Principal then makes the final decision, resulting in APPROVED or REJECTED. All state transitions are validated server-side and return HTTP 400 for invalid transitions.
7. File storage for notes uploads, submission files, club logos, and event posters is handled by Firebase Storage through the FirebaseStorageService. Files are uploaded to a configured bucket and the returned public URL is persisted in the respective database entity.
8. The Broadcast module delivers scoped announcements at CAMPUS, COLLEGE, or DEPARTMENT level. The BroadcastServiceImpl queries the student's college and department context to filter relevant broadcasts, and the senderRole field in BroadcastResponse provides attribution (e.g., "Campus Admin — Campus-wide").
9. The Timetable module allows HODs to create, update, and delete weekly schedule entries scoped to a batch, with teacher and room allocation. Students retrieve their department's timetable through GET /api/student/timetable, filtered by their enrolled department.
10. Email notifications for registration confirmations, club approvals, and broadcast alerts are dispatched asynchronously using Spring Mail with @Async to avoid blocking the HTTP request thread, ensuring API response times remain low regardless of mail server latency.

4.2.2 Major Classes

1. AuthController and AuthService Class

- Handles all authentication endpoints: POST /api/auth/register, /login, /refresh, and /logout
- Implements BCrypt password hashing (strength factor 10) for secure credential storage
- Issues signed JWT access tokens and refresh tokens via JwtUtil, and records invalidated JTIs in the InvalidatedToken table on logout
- Exposes public endpoints GET /api/auth/colleges and /api/auth/departments/{collegeId} for registration dropdowns, requiring no authentication

2. JwtAuthFilter and SecurityConfig Class

- JwtAuthFilter intercepts every HTTP request, extracts the Bearer token, validates its signature and expiry, and checks the InvalidatedToken table for blacklisted JTIs
- Populates the Spring SecurityContext with the authenticated user's role so that downstream controllers can enforce method-level access without redundant database calls

- SecurityConfig defines URL-pattern role rules (e.g., /api/admin/** → CAMPUS_ADMIN only) and configures stateless session management, CORS policies, and CSRF exemption for REST clients
- CorsConfig provides the highest-priority CorsFilter bean to handle CORS preflight OPTIONS requests before Spring Security processes them, preventing 403 errors on browser preflight checks

3. CampusAdminController Class

- Exposes 13 endpoints under /api/admin for managing colleges, seminar halls, campuslevel events, broadcasts, and the system dashboard
- Delegates college lifecycle operations (create, approve, status update, principal assignment) to CollegeServiceImpl, which auto-generates a unique college code using UUID prefix logic
- Manages PUBLIC-type seminar halls accessible across all colleges, and creates CAMPUS-level events visible to all students system-wide
- Returns dashboard statistics (total colleges, users, events, registrations, clubs, active students) via DashboardServiceImpl using count queries across all entity repositories

4. PrincipalController Class

- Exposes 19 endpoints under /api/principal for department management, professor listing, club request review, seminar hall management, college-level events, and broadcasts
- Automatically injects the authenticated principal's college context into DepartmentCreateRequest, removing the need for the frontend to pass collegeId explicitly and preventing cross-college data manipulation
- Enforces COLLEGE-level event creation by overriding the eventLevel field in EventCreateRequest before delegating to EventServiceImpl, ensuring principals cannot create campus-wide events
- Supports status-filtered club request listing (GET /api/principal/clubrequests?status=PENDING_PRINCIPAL), enabling the frontend to show only actionable items

5. HODController Class

- Exposes 17 endpoints under /api/hod for department-level events, club request management, timetable CRUD, seminar hall creation, and department broadcasts

- TimetableServiceImpl creates entries scoped to a specific batch and teacher; the HOD's department context is auto-derived from the authenticated user's profile, preventing cross-department timetable creation
- Enforces DEPARTMENT-level event creation, ensuring HODs cannot create collegewide or campus-wide events
- Supports status-filtered club request listing (GET /api/hod/clubrequests?status=PENDING_HOD) so HODs only see requests awaiting their action

6. ProfessorController Class

- Exposes 19 endpoints under /api/professor for batch management, section creation, submission review, notes upload, teacher availability, event management, and student progress tracking
- BatchServiceImpl auto-assigns the professor's department to each new batch (departmentId is derived from the authenticated user entity), removing the need for manual department selection in the frontend form
- Submission review endpoint (PUT /api/professor/submissions/{id}/remark) accepts a SubmissionRemarkRequest with a remark string and a SubmissionStatus enum value (UNDER_REVIEW, APPROVED, REJECTED), updating the submission record and recording the reviewing professor's identity
- Teacher availability slots (POST /api/professor/availability) are persisted with date, from/to time, status (AVAILABLE, BUSY, ON_LEAVE), and a note—making them visible to students via GET /api/student/teacher-availability

7. StudentController Class

- Exposes 16 endpoints under /api/student covering events, clubs, submissions, notes, broadcasts, timetable, teacher availability, progress, sections, and profile updates
- GET /api/student/events returns a filtered list of EventResponse objects relevant to the student's college and department, with an isRegistered flag computed per event so the frontend can show Register / Already Registered without additional API calls
- POST /api/student/events/{id}/register triggers EventRegistrationServiceImpl, which checks capacity ($R(e) < C(e)$), creates an EventRegistration record, and for paid events creates a Stripe PaymentIntent returning the client secret
- GET /api/student/sections returns BatchSection records relevant to the student's department, enabling the Submissions page to populate the section selector dynamically

8. EventServiceImpl and EventRegistrationServiceImpl Class

- EventServiceImpl implements event-level auto-assignment and validation: CAMPUS_ADMIN events are forced to CAMPUS level; PRINCIPAL events to COLLEGE level; HOD events to DEPARTMENT level—enforced in the respective controllers before service delegation
- getVisibleEventsForStudent queries events matching the student's campus, college, or department scope and maps each to EventResponse with a participantCount and isRegistered boolean computed from EventRegistration records
- EventRegistrationServiceImpl.registerForEvent validates event capacity, creates a ticket with a UUID-based ticketCode, and conditionally calls StripeService.createPaymentIntent when ticketPrice > 0, returning the Stripe client secret in EventRegistrationResponse
- getParticipants returns a list of EventParticipantResponse objects (student name, email, registration date, ticket status) used by admin, principal, HOD, and professor event management pages

9. ClubServiceImpl Class

- Manages the full club lifecycle: create (sets status to PENDING_HOD), approveByHOD (advances to PENDING_PRINCIPAL), rejectByHOD (sets REJECTED), approveByPrincipal (sets APPROVED), rejectByPrincipal (sets REJECTED), and joinClub (adds student to members collection)
- mapToResponse populates the isMember flag (true if the current authenticated user's ID is in the club's members set) and the isOwner flag (true if the current user created the club), enabling the frontend to conditionally render Join / Already a Member / Owner badges
- getClubRequests accepts an optional ClubStatus query parameter to filter results; HOD endpoint filters for PENDING_HOD clubs in their department; Principal endpoint filters for PENDING_PRINCIPAL clubs in their college
- College context is auto-injected from the authenticated user's profile in both HOD and Principal controllers, preventing cross-college club data leakage

10. FirebaseStorageService, FCMService, and StripeService Class

- FirebaseStorageService.uploadFile accepts a MultipartFile, streams it to the configured Firebase Storage bucket under a timestamped path, and returns the public download URL to be stored in the corresponding entity field
- FCMService.sendToToken dispatches a targeted FCM push notification to a specific device token (stored in the User entity via PUT /api/student/profile with an fcmToken field); sendToTopic broadcasts to a college or department topic

- StripeService uses the stripe-java v31 StripeClient API to create PaymentIntents with the event's ticketPrice in paise/cents and returns the client secret; the WebhookController verifies the Stripe-Signature header using the webhook secret before processing payment_intent.succeeded events
- All three services are injected as Spring-managed beans; FirebaseStorageService and FCMService depend on the FirebaseApp bean initialized in FirebaseConfig from the service account JSON file at classpath:firebase-service-account.json

These classes modularize the system functionality across well-defined layers, enabling efficient parallel development, independent unit testing, and future scalability of CampusNexus's roledriven campus administration services. Each class adheres to the single-responsibility principle, communicates exclusively through defined service interfaces, and returns standardized ApiResponse<T> wrappers to maintain a consistent contract between the backend and the React frontend.

4.2.3 Code Listing

```
package com.campusnexus.service.impl;

import com.campusnexus.dto.*; import com.campusnexus.entity.InvalidatedToken; import
com.campusnexus.entity.User; import com.campusnexus.enums.Role; import
com.campusnexus.exception.BadRequestException; import
com.campusnexus.exception.DuplicateResourceException; import
com.campusnexus.exception.ResourceNotFoundException; import
com.campusnexus.repository.CollegeRepository; import
com.campusnexus.repository.DepartmentRepository; import
com.campusnexus.repository.InvalidatedTokenRepository; import
com.campusnexus.repository.UserRepository; import com.campusnexus.security.JwtUtil;
import com.campusnexus.service.AuthService; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.core.Authentication; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.security.core.userdetails.UserDetailsService; import
```

```
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.stereotype.Service; import
org.springframework.transaction.annotation.Transactional;
```

```
@Service public class AuthServiceImpl implements
AuthService {    private final UserRepository
userRepository;    private final CollegeRepository
collegeRepository;    private final
DepartmentRepository departmentRepository;    private
final InvalidatedTokenRepository
invalidatedTokenRepository;    private final
PasswordEncoder passwordEncoder;    private final
JwtUtil jwtUtil;    private final AuthenticationManager
authenticationManager;    private final
UserDetailsService userDetailsService;

    public AuthServiceImpl(UserRepository userRepository,
                           CollegeRepository collegeRepository,
                           DepartmentRepository departmentRepository,
                           InvalidatedTokenRepository invalidatedTokenRepository,
                           PasswordEncoder passwordEncoder,
                           JwtUtil jwtUtil,
                           AuthenticationManager authenticationManager,
                           UserDetailsService userDetailsService) {        this.userRepository =
userRepository;        this.collegeRepository = collegeRepository;
this.departmentRepository = departmentRepository;
this.invalidatedTokenRepository = invalidatedTokenRepository;
this.passwordEncoder = passwordEncoder;        this.jwtUtil =
jwtUtil;        this.authenticationManager = authenticationManager;
this.userDetailsService = userDetailsService;
    }
```

```
@Override @Transactional public LoginResponse
register(RegisterRequest request) {    if
(request.getRole() == Role.CAMPUS_ADMIN) {
throw new BadRequestException("Cannot register with
CAMPUS_ADMIN role");
}

    if (userRepository.existsByEmail(request.getEmail())) {        throw new
DuplicateResourceException("Email already registered: " + request.getEmail());
    }

    User user = User.builder()
        .name(request.getName())
        .email(request.getEmail())
        .password(passwordEncoder.encode(request.getPassword()))
        .phone(request.getPhone())
        .role(request.getRole())
        .isActive(true)
        .build();

    if (request.getCollegeId() != null) {
user.setCollege(collegeRepository.findById(request.getCollegeId())
        .orElseThrow(() -> new ResourceNotFoundException("College not found")));
    }

    if (request.getDepartmentId() != null) {
user.setDepartment(departmentRepository.findById(request.getDepartmentId())
        .orElseThrow(() -> new ResourceNotFoundException("Department not found")));
    }
```

```
        userRepository.save(user);

        UserDetails userDetails = userDetailsService.loadUserByUsername(user.getEmail());
        String accessToken = jwtUtil.generateAccessToken(userDetails);
        String refreshToken = jwtUtil.generateRefreshToken(user.getEmail());

        return LoginResponse.builder()
            .accessToken(accessToken)
            .refreshToken(refreshToken)
            .role(user.getRole())
            .userId(user.getId())
            .name(user.getName())
            .email(user.getEmail())
            .build();
    }

    @Override    public LoginResponse
    login(LoginRequest request) {
        Authentication authentication = authenticationManager.authenticate(            new
        UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
        );

        UserDetails userDetails = (UserDetails) authentication.getPrincipal();
        User user = userRepository.findByEmail(request.getEmail())
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));

        String accessToken = jwtUtil.generateAccessToken(userDetails);
        String refreshToken = jwtUtil.generateRefreshToken(user.getEmail());

        return LoginResponse.builder()
            .accessToken(accessToken)
```

```
.refreshToken(refreshToken)

.role(user.getRole())

.userId(user.getId())

.name(user.getName())

.email(user.getEmail())

.build();
}
```

```
@Override public LoginResponse
refreshToken(RefreshTokenRequest request) {

    String refreshToken = request.getRefreshToken();

    if (invalidatedTokenRepository.existsByToken(refreshToken)) {
        throw new BadRequestException("Refresh token has been invalidated");
    }

    if (jwtUtil.isTokenExpired(refreshToken)) {        throw new
        BadRequestException("Refresh token has expired");
    }

    String email = jwtUtil.extractEmail(refreshToken);
    UserDetails userDetails = userDetailsService.loadUserByUsername(email);
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found"));

    String newAccessToken = jwtUtil.generateAccessToken(userDetails);
    String newRefreshToken = jwtUtil.generateRefreshToken(email);

    // Invalidate old refresh token

    InvalidatedToken invalidated = InvalidatedToken.builder()
        .token(refreshToken)
```

```
.build();

invalidatedTokenRepository.save(invalidated);

return LoginResponse.builder()
    .accessToken(newAccessToken)
    .refreshToken(newRefreshToken)
    .role(user.getRole())
    .userId(user.getId())
    .name(user.getName())
    .email(user.getEmail())
    .build();
}

@Override @Transactional public void
logout(String token) {    if (token != null &&
token.startsWith("Bearer ")) {        token =
token.substring(7);
    }

    if (token != null && !invalidatedTokenRepository.existsByToken(token)) {
        InvalidatedToken invalidated = InvalidatedToken.builder()
            .token(token)
            .build();
        invalidatedTokenRepository.save(invalidated);
    }
}

package com.campusnexus.controller;
```

```
import com.campusnexus.dto.*; import
com.campusnexus.entity.College; import
com.campusnexus.entity.Department; import
com.campusnexus.repository.CollegeRepository; import
com.campusnexus.repository.DepartmentRepository;
import com.campusnexus.service.AuthService; import
io.swagger.v3.oas.annotations.Operation; import
io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.validation.Valid; import
org.springframework.http.ResponseEntity; import
org.springframework.web.bind.annotation.*;

import java.util.List; import
java.util.UUID; import
java.util.stream.Collectors;

@RestController
@RequestMapping("/api/auth")
@Tag(name = "Auth", description = "Authentication endpoints")
@CrossOrigin(origins = "*", allowedHeaders = "*") public class
AuthController {

    private final AuthService authService;    private final
CollegeRepository collegeRepository;    private final
DepartmentRepository departmentRepository;

    public AuthController(AuthService authService,
                           CollegeRepository collegeRepository,
                           DepartmentRepository departmentRepository) {
```

```
this.authService = authService;    this.collegeRepository =
collegeRepository;    this.departmentRepository =
departmentRepository;
}

@PostMapping("/register")

@Operation(summary = "Register a new user", description = "Register a new user with any role
except CAMPUS_ADMIN")

@ApiResponses({

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description =
"Registration successful"),

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "400", description =
"Invalid input"),

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "409", description =
"Email already exists")

})

public ResponseEntity<ApiResponse<LoginResponse>> register(@Valid @RequestBody
RegisterRequest request) {

    LoginResponse response = authService.register(request);    return
ResponseEntity.ok(ApiResponse.success("Registration successful", response));

}

@PostMapping("/login")

@Operation(summary = "Login", description = "Authenticate and get JWT tokens")

@ApiResponses({

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description =
"Login successful"),

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "401", description =
"Invalid credentials")

})

public ResponseEntity<ApiResponse<LoginResponse>> login(@Valid @RequestBody
LoginRequest request) {

    LoginResponse response = authService.login(request);    return
ResponseEntity.ok(ApiResponse.success("Login successful", response));

}
```

```
}

@PostMapping("/refresh")

@Operation(summary = "Refresh token", description = "Get a new access token using refresh token")

@ApiResponses({

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description = "Token refreshed"),

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "400", description = "Invalid refresh token")

})

public ResponseEntity<ApiResponse<LoginResponse>> refreshToken(@Valid @RequestBody RefreshTokenRequest request) {

    LoginResponse response = authService.refreshToken(request);    return

    ResponseEntity.ok(ApiResponse.success("Token refreshed", response));

}

@PostMapping("/logout")

@Operation(summary = "Logout", description = "Invalidate current token")

@ApiResponses({

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description = "Logged out successfully")

})

public ResponseEntity<ApiResponse<Void>> logout(HttpServletRequest request) {

    String authHeader = request.getHeader("Authorization");

    authService.logout(authHeader);    return

    ResponseEntity.ok(ApiResponse.success("Logged out successfully"));

}

@GetMapping("/colleges")

@Operation(summary = "Get all active colleges", description = "Public endpoint to fetch colleges for registration dropdown")

@ApiResponses({
```

```
@io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description =
"Colleges retrieved")
})

public ResponseEntity<ApiResponse<List<CollegeDropdownResponse>>> getColleges() {
List<CollegeDropdownResponse> colleges = collegeRepository.findAll()

    .stream()

    .filter(c -> c.getStatus() == com.campusnexus.enums.CollegeStatus.ACTIVE)

    .map(c -> CollegeDropdownResponse.builder()

        .id(c.getId())

        .name(c.getName())

        .city(c.getCity())

        .build())

    .collect(Collectors.toList());    return

ResponseEntity.ok(ApiResponse.success("Colleges retrieved", colleges));
}
```

```
@GetMapping("/departments/{collegeId}")

@Operation(summary = "Get departments by college", description = "Public endpoint to fetch
departments for registration dropdown")

@ApiResponses({

    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description =
"Departments retrieved")

})

public ResponseEntity<ApiResponse<List<DepartmentDropdownResponse>>>
getDepartmentsByCollege(

    @PathVariable UUID collegeId) {

    List<DepartmentDropdownResponse> departments =
departmentRepository.findByCollegeId(collegeId)

    .stream()

    .map(d -> DepartmentDropdownResponse.builder()

        .id(d.getId())

        .name(d.getName())

        .code(d.getCode())
```

```
        .build()

        .collect(Collectors.toList());

        return ResponseEntity.ok(ApiResponse.success("Departments retrieved", departments));
    }
}
```

```
package com.campusnexus.repository;
```

```
import com.campusnexus.entity.User; import
com.campusnexus.enums.Role; import
org.springframework.data.jpa.repository.JpaRepository; import
org.springframework.stereotype.Repository; import
org.springframework.data.jpa.repository.Query; import
org.springframework.data.repository.query.Param;
```

```
import java.util.List; import
java.util.Optional; import
java.util.UUID;
```

```
@Repository public interface UserRepository extends
```

```
JpaRepository<User, UUID> {
```

```
    Optional<User> findByEmail(String email);
```

```
    boolean existsByEmail(String email);
```

```
    List<User> findByRoleAndCollegeId(Role role, UUID collegeId);
```

```
    List<User> findByRoleAndDepartmentId(Role role, UUID departmentId);
```

```
    List<User> findByCollegeId(UUID collegeId);
```

```
List<User> findByRole(Role role);
```

```
long countByRole(Role role);
```

```
long countByRoleAndIsActiveTrue(Role role);
```

```
@Query("SELECT u FROM User u WHERE " +  
        "LOWER(u.name) LIKE LOWER(CONCAT('%', :query, '%')) OR " +  
        "LOWER(u.email) LIKE LOWER(CONCAT('%', :query, '%')) OR " +  
        "u.phone LIKE CONCAT('%', :query, '%')")  
List<User> searchByNameEmailOrPhone(@Param("query") String query); }
```

```
-----  
-----  
  
import { useCallback } from 'react' import { useNavigate } from  
'react-router-dom' import useAuthStore from '../store/authStore'  
import { authAPI } from '../api/auth.api' import {  
ROLE_DASHBOARD_PATHS } from '../utils/constants'  
import toast from 'react-hot-toast'
```

```
export function useAuth() { const navigate = useNavigate() const { user,  
isAuthenticated, login: storeLogin, logout: storeLogout } = useAuthStore()
```

```
const login = useCallback(  
async (email, password) => {  
  try {  
    const res = await authAPI.login({ email, password })  
    const data = res.data.data || res.data
```

```
const userData = {      id:
data.userId || data.id,      name:
data.name,      email: data.email,
role: data.role,      collegeId:
data.collegeId,      departmentId:
data.departmentId,      profilePicUrl:
data.profilePicUrl,
}

storeLogin(userData, data.accessToken, data.refreshToken)
toast.success(`Welcome back, ${userData.name}!`)

const dashboardPath = ROLE_DASHBOARD_PATHS[userData.role] || '/login'
navigate(dashboardPath, { replace: true })

return { success: true }    } catch (error) {      const message =
error.response?.data?.message || 'Login failed. Please try again.'
toast.error(message)      return { success: false, error: message }
}
},
[storeLogin, navigate]
)

const register = useCallback(
async (formData) => {
  try {
    await authAPI.register(formData)

    toast.success('Registration successful! Please login.')      navigate('/login', { replace:
true })      return { success: true }    } catch (error) {      const message =
```

```
error.response?.data?.message || 'Registration failed. Please try again.'
toast.error(message)    return { success: false, error: message }
    }
  },
  [navigate]
)
```

```
const logout = useCallback(async () => {
  try {    await
authAPI.logout()
    } catch {
      // Ignore logout API errors
    } finally {    storeLogout()
toast.success('Logged out successfully')
navigate('/login', { replace: true })
    }
  }, [storeLogout, navigate])
```

```
    return {    user,
isAuthenticated,
role: user?.role,
login,    register,
logout,

    }
}
```

```
export default useAuth
```

```
import { useState, useEffect } from 'react' import { Link, useLocation
} from 'react-router-dom' import { motion } from 'framer-motion'
```

```
import { useForm } from 'react-hook-form' import { zodResolver }
from '@hookform/resolvers/zod' import { z } from 'zod' import {
Mail, Lock, GraduationCap, ArrowRight } from 'lucide-react' import
toast from 'react-hot-toast' import Input from
'../../components/ui/Input' import Button from
'../../components/ui/Button' import useAuth from '../../hooks/useAuth'
import { ROLE_LABELS } from '../../utils/constants'

const loginSchema = z.object({ email: z.string().email('Please enter
a valid email'), password: z.string().min(6, 'Password must be at
least 6 characters'),
})

const roles = ['CAMPUS_ADMIN', 'PRINCIPAL', 'HOD', 'PROFESSOR', 'STUDENT']

export default function LoginPage() { const [selectedRole,
setSelectedRole] = useState('STUDENT') const [loading,
setLoading] = useState(false) const { login } = useAuth()
const location = useLocation()

// Show toast if redirected due to role change
useEffect(() => { const params = new
URLSearchParams(location.search) const reason =
params.get('reason') if (reason === 'role_changed') {
toast.error(
  'Your role has been changed by the administrator. Please login again to continue.',
  { duration: 6000, id: 'role-changed' }
)
}
}, [])
```

```

const { register, handleSubmit,
formState: { errors }, } = useForm({
resolver: zodResolver(loginSchema),
defaultValues: { email: "", password: "" },
})

```

```

const onSubmit = async (data) => {
setLoading(true)  await
login(data.email, data.password)
setLoading(false)
}

```

```

const isRoleChanged = new URLSearchParams(location.search).get('reason') === 'role_changed'

```

```

return (
<div className="min-h-screen flex">
  {/* Left Panel — Gradient */}
  <div className="hidden lg:flex lg:w-1/2 gradient-bg relative overflow-hidden items-center justify-center p-12">
    <div className="absolute inset-0 mesh-gradient opacity-30" />
    <motion.div      animate={{ y: [0, -20, 0] }}      transition={{
duration: 6, repeat: Infinity, ease: 'easeInOut' }}      className="absolute
top-20 left-16 text-accent-400 text-4xl opacity-40"
    >
      ★
    </motion.div>    <motion.div      animate={{ y: [0, -15, 0] }}
transition={{ duration: 7, repeat: Infinity, ease: 'easeInOut', delay: 1 }}
className="absolute bottom-32 right-16 text-accent-400 text-3xl opacity-50"
    >
      ◆
    </motion.div>

```

```

<div className="relative z-10 text-center">

  <motion.div      initial={{
opacity: 0, scale: 0.8 }}
animate={{ opacity: 1, scale: 1 }}
transition={{ duration: 0.6 }}

    className="w-24 h-24 rounded-3xl bg-white/10 backdrop-blur flex items-center justifycenter
mx-auto mb-8"

    >

    <GraduationCap className="w-12 h-12 text-white" />

  </motion.div>

<motion.h2      initial={{
opacity: 0, y: 20 }}
animate={{ opacity: 1, y: 0 }}
transition={{ delay: 0.2 }}
className="text-4xl font-bold font-
heading text-white mb-4"

  >

    Welcome to <span className="text-accent-400">CampusNexus</span>

  </motion.h2>    <motion.p      initial={{
opacity: 0, y: 20 }}      animate={{ opacity: 1, y: 0
}}      transition={{ delay: 0.3 }}
className="text-white/60 max-w-sm mx-auto"

  >

    Your complete campus management solution. Login to access your personalized dashboard.

</motion.p>

  <motion.div      initial={{
opacity: 0, x: -30 }}
animate={{ opacity: 1, x: 0 }}
transition={{ delay: 0.5 }}

```

```
className="mt-12 bg-white/10 backdrop-blur rounded-2xl p-4 border border-white/20 maxw-
xs mx-auto"
```

```
>
```

```
<div className="flex items-center gap-3">
```

```
<div className="flex -space-x-2">
```

```
{['bg-blue-500', 'bg-purple-500', 'bg-green-500', 'bg-orange-500'].map((c, i) => (
```

```
<div key={i} className={`w-8 h-8 rounded-full ${c} border-2 border-white/20`} />
```

```
)))
```

```
</div>
```

```
<div className="text-left">
```

```
<p className="text-sm font-semibold text-white">2k+ Users</p>
```

```
<p className="text-xs text-white/50">Active on platform</p>
```

```
</div>
```

```
</div>
```

```
</motion.div>
```

```
</div>
```

```
</div>
```

```
{/* Right Panel — Login Form */}
```

```
<div className="flex-1 flex items-center justify-center p-6 sm:p-12 bg-surface-50">
```

```
<motion.div      initial={{
```

```
opacity: 0, x: 30 }}      animate={{
```

```
opacity: 1, x: 0 }}      transition={{
```

```
duration: 0.5 }}
```

```
className="w-full max-w-md"
```

```
>
```

```
{/* Mobile Logo */}
```

```
<div className="lg:hidden flex items-center gap-2 mb-8">
```

```
<div className="w-10 h-10 rounded-xl bg-primary-600 flex items-center justify-center">
```

```
<GraduationCap className="w-6 h-6 text-white" />
```

```
</div>
```

```

    <span className="text-xl font-bold font-heading text-dark-900">
      Campus<span className="text-accent-400">Nexus</span>
    </span>
  </div>

  <h1 className="text-3xl font-bold font-heading text-dark-900 mb-2">Welcome Back</h1>
  <p className="text-dark-500 mb-8">Enter your credentials to access your dashboard</p>

  {/* Role changed warning banner */}
  {isRoleChanged && (
    <motion.div          initial={{
opacity: 0, y: -10 }}
animate={{ opacity: 1, y: 0 }}

      className="mb-6 p-4 bg-amber-50 border border-amber-200 rounded-xl flex items-start gap-
3"
    >
      <div className="w-5 h-5 rounded-full bg-amber-400 flex items-center justify-center
flexshrink-0 mt-0.5">
        <span className="text-white text-xs font-bold">!</span>
      </div>
      <div>
        <p className="text-sm font-semibold text-amber-800">Role Updated</p>
        <p className="text-xs text-amber-600 mt-0.5">
          Your role has been changed by the administrator. Please login again to access your new
          dashboard.
        </p>
      </div>
    </motion.div>
  )}

  {/* Role Tabs */}
  <div className="flex flex-wrap gap-2 mb-8">

```

```

    {roles.map((role) => (
      <button          key={role}
onClick={() => setSelectedRole(role)}

      className={`px-3 py-1.5 rounded-full text-xs font-medium transition-all ${selectedRole
=== role

      ? 'bg-primary-600 text-white shadow-md'
      : 'bg-dark-100 text-dark-500 hover:bg-dark-200'
      }}
    >
      {ROLE_LABELS[role]}
    </button>
  ))}
</div>

```

```

<form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
  <Input          label="Email
Address"          type="email"
icon={Mail}
placeholder="you@example.com"
error={errors.email?.message}
    {...register('email')}
  />
  <Input          label="Password"
type="password"    icon={Lock}
placeholder="Enter your password"
error={errors.password?.message}
    {...register('password')}
  />

```

```

<div className="flex items-center justify-between">
  <label className="flex items-center gap-2 cursor-pointer">

```

```
<input type="checkbox" className="w-4 h-4 rounded border-dark-300 text-primary-600 focus:ring-primary-500" />
<span className="text-sm text-dark-500">Remember me</span>
</label>
<a href="#" className="text-sm text-primary-600 hover:text-primary-700 font-medium">
  Forgot password?
</a>
</div>
```

```
<Button
  type="submit"
  variant="primary"
  size="lg"
  loading={loading}
  className="w-full"
  icon={ArrowRight}
  iconPosition="right"
  >
```

```
  Sign In
</Button>
</form>
```

```
<p className="mt-8 text-center text-sm text-dark-500">
  Don't have an account?{' '}
  <Link to="/register" className="text-primary-600 hover:text-primary-700 font-semibold">
    Create Account
  </Link>
</p>
</motion.div>
</div>
```

</div>

)

}

4.3 Screenshots

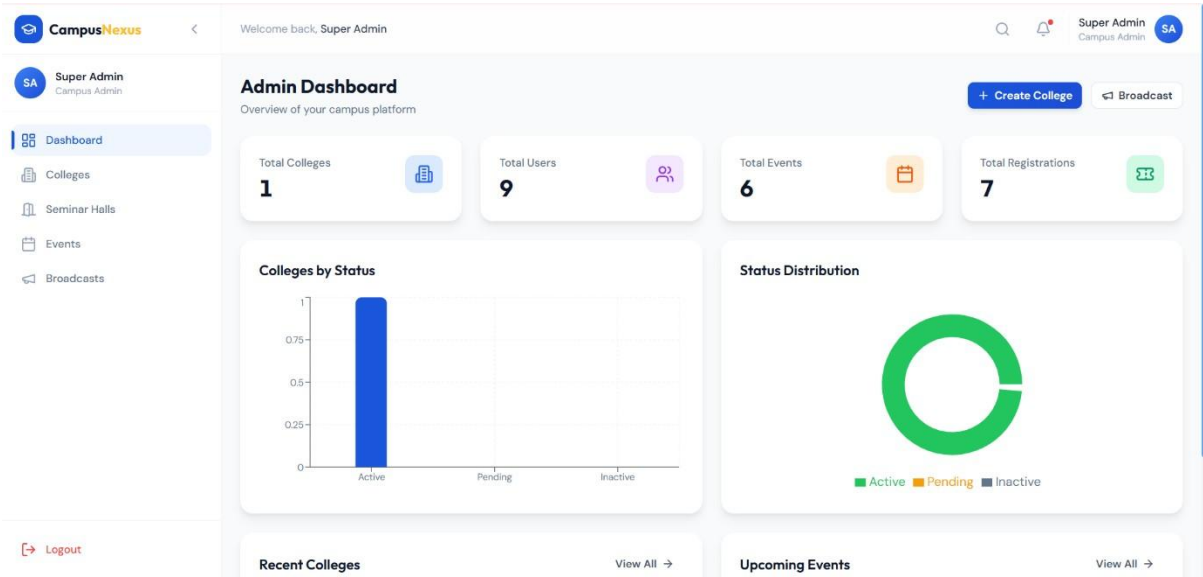


Fig. 4.1 Admin Dashboard

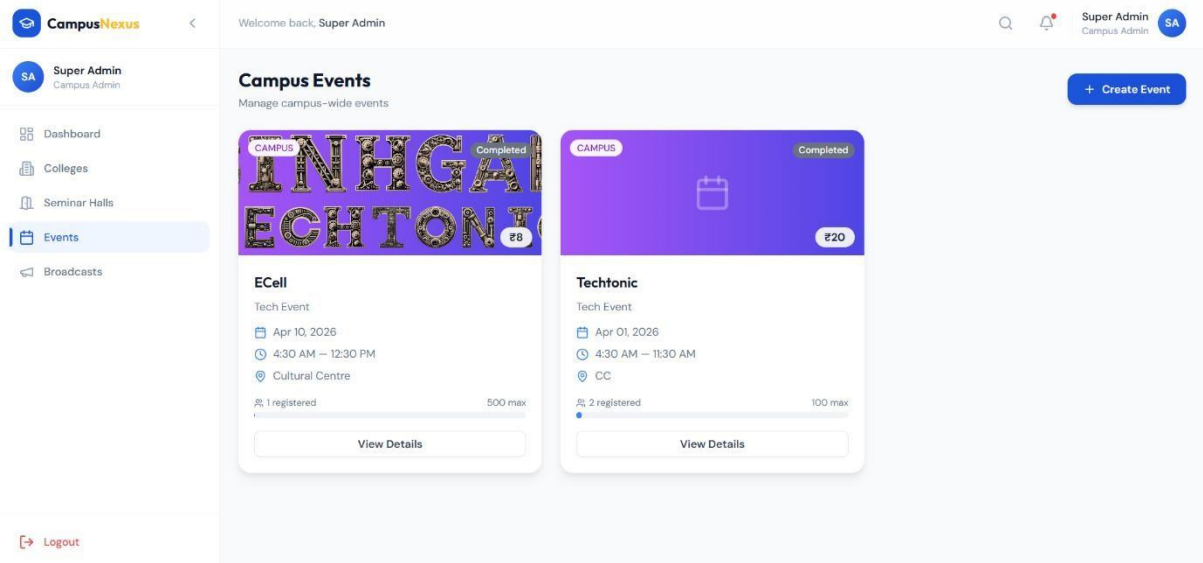


Fig. 4.2 Campus Events

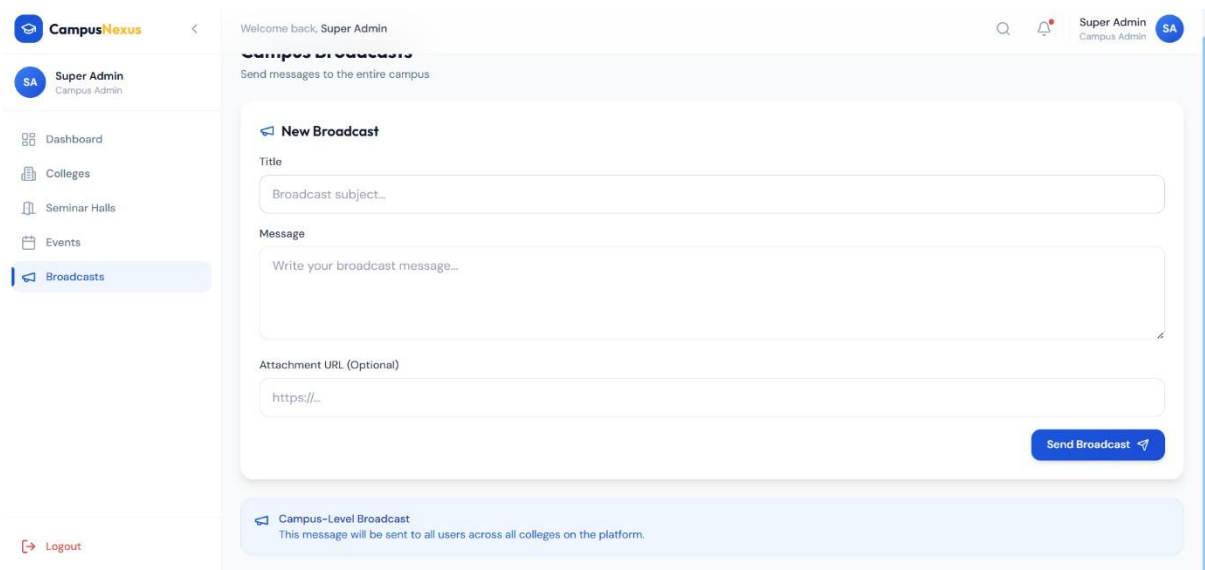


Fig. 4.3 Super Admin Dashboard

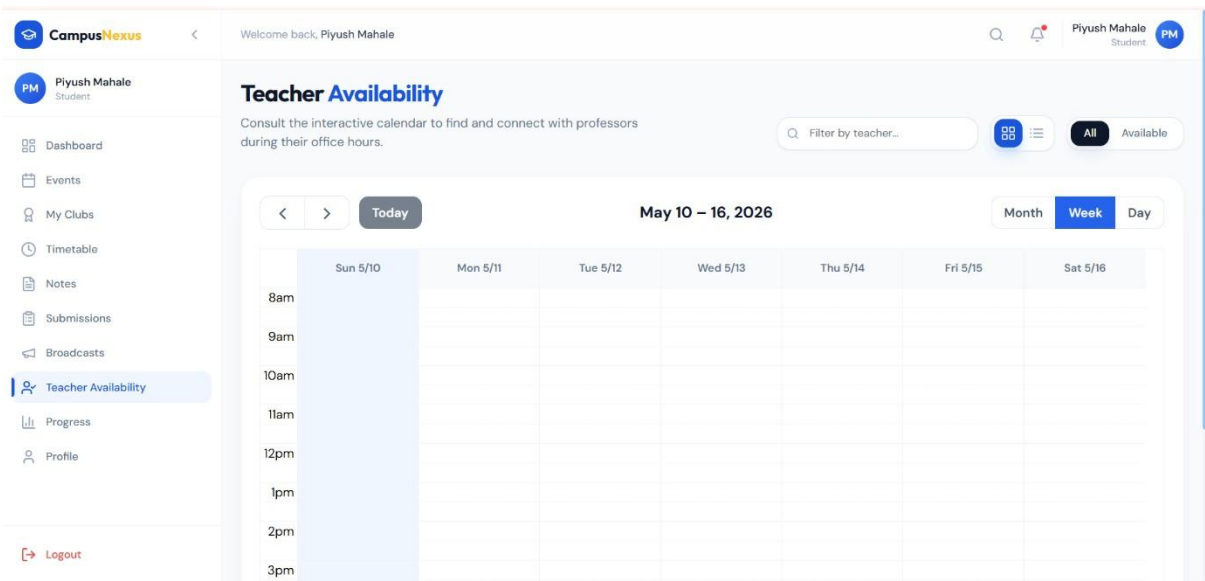


Fig. 4.4 Teacher Availability

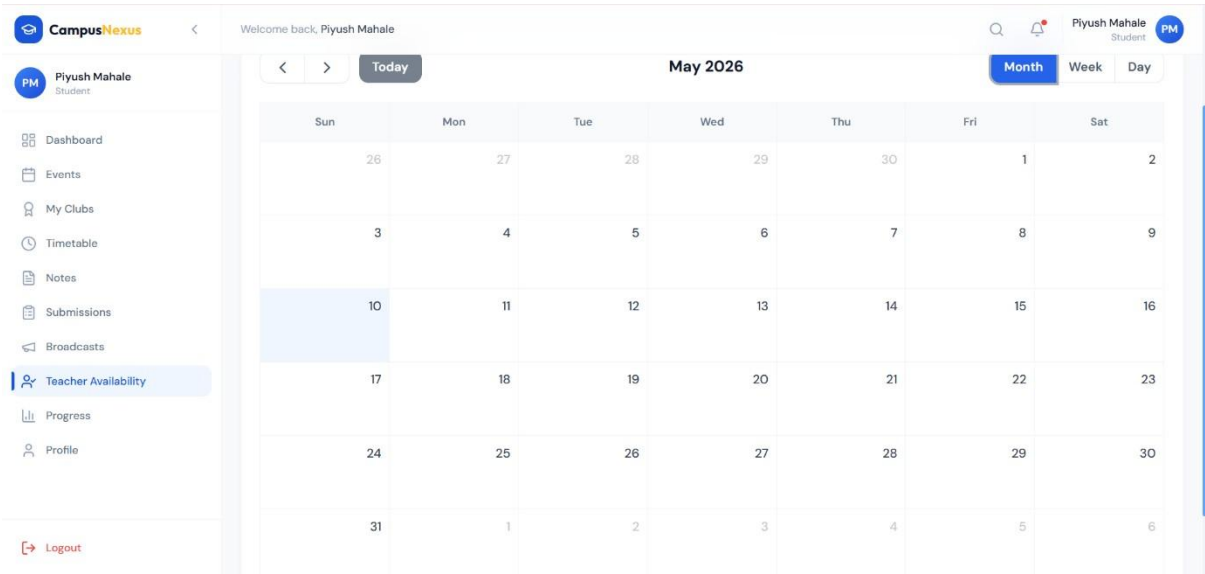


Fig. 4.5 Teacher Availability 2

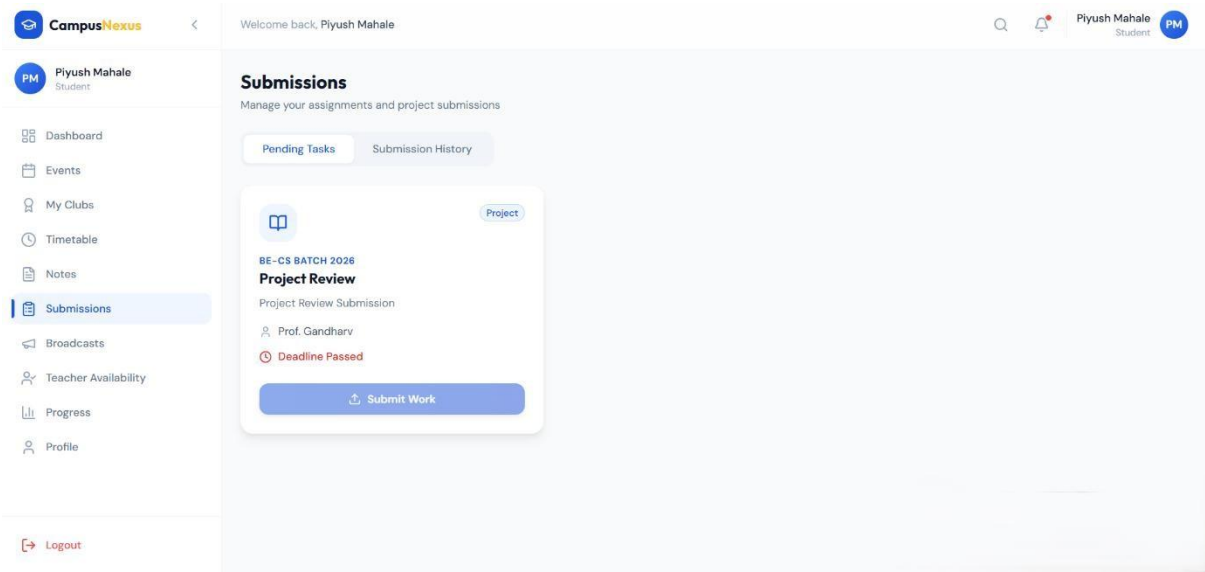


Fig. 4.6 Submissions Dashboard

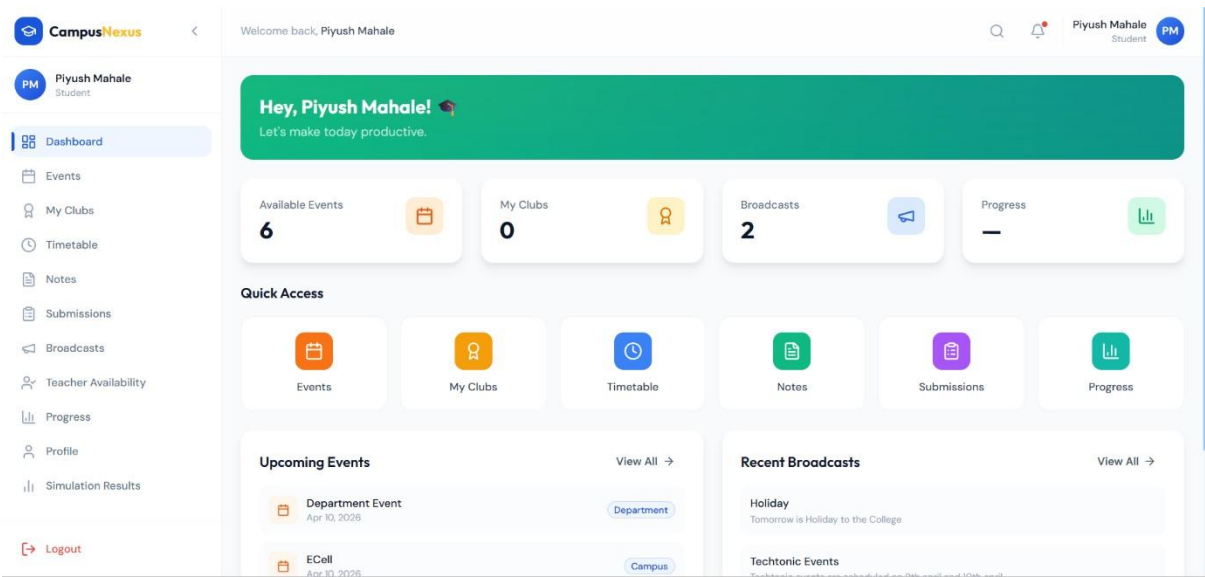


Fig. 4.7 Student Dashboard

Chapter 5

TESTING

5.1 Introduction

Testing is a method of assessing the functionality of a software program. This chapter covers the testing approach used and the test cases.

5.2 Unit Testing

Unit Testing is a level of software testing where individual units/components of a software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software (e.g., a function, a class method, or a component).

Table 4.1 : Unit Testing

Sr. No.	Module	Description	Expected Output	Actual Output	Result
1	Role/Hierarchy Management (Function: assignPrincipal ())	Campus Admin assigns the Principal role to a professor for a new college.	Professor's access profile should be updated to Principal ; a new college record should be created.	Professor's profile updated to Principal; College record created.	Pass
		Campus Admin attempts to assign the Principal role to a user not registered as a Professor.	System should return an error: "Invalid User Role for Assignment."	System returned error: "Invalid User Role for Assignment."	Pass
		Principal attempts to assign a Professor as HOD of a specific department.	Professor's profile updated to HOD for the specified department; Principal retains all their own permissions.	Professor's profile updated; Principal permissions maintained.	Pass

2	Workflow Approval (Function: initiateClubRequest())	Student submits a Club Creation Request.	Request status set to ' Pending Professor Endorsement '; an automatic notification sent	Status set to 'Pending Professor Endorsement'; Notification sent successfully.	Pass
---	--	--	--	--	------

			to the selected Professor.		
	(Function: updateWorkflowStatus())	HOD approves a pending Club Request (next step is Principal Approval).	Request status updated to ' Pending Principal Approval '; HOD's dashboard task list should be updated (removed).	Status updated to 'Pending Principal Approval'; Task removed from HOD list.	Pass
		HOD attempts to approve a request rejected by the Professor/Guide.	System should prevent approval and return error: "Cannot proceed, request was previously rejected."	System prevented approval and returned error message.	Pass
3	Academic Progress Tracker (Function: submitSubmission())	Student uploads a large Project file (e.g., 50MB) for team submission.	File successfully stored in Cloud Storage; submission record created and linked to all team members.	File stored; Record created and linked to team.	Pass
	(Function: saveProfessorRemarks())	Professor submits remarks and the ' Pass ' status for a student's Seminar section.	Student's progress record updated with 'Pass' status and remarks; remarks should be instantaneously visible on the student's dashboard.	Status updated and remarks visible on student dashboard immediately.	Pass
		Professor attempts to save remarks without entering text (mandatory field).	System should prompt: "Remarks field is mandatory for status update."	System prompted for mandatory remarks field.	Pass

4	Notes/Timetable Management (Function: uploadNotes())	Professor attempts to upload a document as "Notes" for a specific Subject/Year that is not in their teaching load.	System should deny the upload with an access violation error.	System denied upload with access violation error.	Pass
	(Function: checkAvailability())	Student attempts to view the availability of a Professor.	Professor's TimeBased Availability schedule should be displayed.	Professor's availability schedule displayed.	Pass

5.3 Integration Testing

Integration testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units (modules).

Table 4.2 : Integration Testing

Sr. No.	Module	Description	Expected Output	Actual Output	Result
1	Role/Hierarchy Management	When Campus Admin assigns the Principal role to a Professor.	Professor's Authentication credentials should sync with the Role Management module, granting access to Principal-specific dashboards (e.g., 'Add Department').	Authentication service successfully grants Principal role access.	Pass

2	Workflow: Club Approval	Student submits a Club Request, and the Professor/Guide endorses it.	The system's Workflow Engine should successfully pass the request object from the Professor's pending task list to the HOD's pending task list (sequential routing).	Request object routed from Professor to HOD task list correctly.	Pass
3	Academic Tracking	Professor submits remarks and 'Pass' status for a student's Project submission.	The data should be written to the Academic Database and simultaneously trigger an update notification on	Database write is successful; Student dashboard receives real-time update.	Pass
			the Student's dashboard module.		
4	Event/Payment Gateway	Student completes the payment for an event ticket via the integrated Payment Gateway.	The Payment Confirmation module should trigger the Ticket Generation service, and the generated ticket should be linked to the student's profile.	Payment confirmed; Ticket generated and linked to the student's profile record.	Pass

5.4 Acceptance Testing

Acceptance Testing is a level of software testing where a system is tested for acceptability. The purpose of this testing is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery. It is the formal testing with respect to user needs, requirements, and business processes, conducted to determine whether or not a system satisfies the acceptance criteria.

Table 4.3 : Acceptance Testing

Sr. No.	Requirement	Description	Expected Output	Actual Output	Result
1	Multi-Level Role Assignment	Campus Admin attempts to assign a Principal to College 'C1', and later the Principal attempts to assign an HOD to Department 'D1'.	Both role assignments must be successful; the assigned HOD should gain access to Department 'D1' management features and no others.	Role assignments successful; HOD access correctly scoped to 'D1'.	Pass
2	Workflow-Enabled Approval	Student submits a Club Request (Sequential Workflow).	The request should only become 'Approved' after receiving acceptance from the Professor, HOD, AND Principal (in sequence).	Request reached final 'Approved' status only after all three sequential acceptances.	Pass
3	Three-Section Progress Tracker	Professor uses the platform to give remarks for all members of a team project.	Remarks and status should be accurately recorded and reflected on the personal progress dashboards of all individual students in the team.	Remarks and status accurately reflected across all team members' dashboards.	Pass
4	Event Ticket/Payment	Student completes the event registration by buying a ticket via the system.	A unique digital Ticket should be generated for the student only after successful payment confirmation is received from the gateway.	Ticket generated successfully upon payment completion.	Pass

5	Teacher Availability Marking	Professor marks their availability status for a particular time slot (Time-Based Teacher Availability Marking).	The marked time slot should be instantly inaccessible for booking by students and visible to the HOD/Principal in a dedicated availability report.	Time slot correctly marked and instantly reflected across student and admin interfaces.	Pass
---	------------------------------	---	--	---	------

Chapter 6

CONCLUSION

The Campus Management System (CMS) platform was successfully developed to address the fragmentation and inefficiency identified in traditional campus administrative and academic monitoring processes. The system achieved its core aim of providing a single, unified digital solution for complex institutional operations. It effectively solved key issues by eliminating paper-based delays through the workflow-enabled approval system for club and event requests, thereby improving administrative efficiency and transparency. Furthermore, the system resolves the problem of inconsistent academic oversight by incorporating the Three-Section Academic Progress Tracker, centralizing student submissions and enabling Professors to give remarks directly on the platform, ensuring no academic data is lost. Crucially, the platform features a Multi-Level Role Assignment hierarchy, providing robust security and control over college and department management. The

application, therefore, ensures the ease of access and operation for all users—from the Campus Admin monitoring data to the Student viewing their real-time academic progress.

REFERENCES

- 1] M. Altayar, N. Ben Fairweather and N. McBride, “Identifying Barriers and Challenges Associated with Campus Portals Adoption: A Comparative Case Study of Saudi and U.K. Universities,” in *Proc. 6th Int. Conf. Internet Technology and Secured Transactions (ICITST)*, 2010. [SciTePress](#)
- 2] P. Subha, I. Fefina, C. Niranjana Devi, S. Suruthika and K. Sushmeena, “College Management System,” *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, vol. 11, no. 5, 2022. [Peer-reviewed Journal](#)
- 3] R. Jain, A. Modi, I. Kashyap, V. Kate and R. Bahrawat, “Research Paper on College Management System,” *IJRPR*, vol. 4, issue 4, 2022. [IJRPR](#)
- 4] O. A. Adedapo, J. Bello, M. A. Ogunrinde, M. O. Adio and O. A. Oyekola, “Development of a Workflow Management System for Higher Education: Case Study of a Department in a Nigerian University,” *FUOYE Journal of Engineering and Technology*, vol. 8, no. 4, Dec 2023. journal.engineering.fuoye.edu.ng

- 5] “Design of Smart Campus Management System Based on Internet of Things Technology,” *Proc. International Conf.* (year/time unspecified), ResearchGate publication. [ResearchGate](#)
- 6] “Design and Implementation of Student Management System,” *Scientific Reports*, 2024. [Nature](#)
- 7] “Digital Campus Management,” *International Journal of Advanced Research in Computer and Communication Engineering*, 2021. [Peer-reviewed Journal](#)
- 8] “Student Activity Request Approval Management System,” *International Research Journal Web*, 2024. [irjweb.com](#)
- 9] “Narrowband-IoT (NB-IoT) and IoT Use Cases in Universities, Campuses, and Educational Institutions: A Research Analysis,” Lyberius E. F. Taruc & Arvin R. De La Cruz, arXiv preprint, Aug 2024. [arXiv](#)
- 10] “A Step-by-Step Guide to Workflow Automations in Higher Education,” EdTech article, 8 July 2025.